

Mohsin Iban Hossain

AIUB, Computer Networks Notes

COMPUTER NETWORKS

Table of Contents

Sl. No	Topic	Pages
1	Network Models	3-15
2	Application Layer Protocols	16-20
3	Data Link Layer –I	21-34
4	Data Link Layer –II	35-43
5	Routing	44-51
6	DHCP & ARP	52-57
7	IP Addressing –I	58-68
8	IP Addressing –II	69-74
9	VLSM	75-85
10	Practice Problem	86-94

NETWORK MODELS

Layered Tasks

➔ Protocol Layering:

- **Concept of Layers:** Used in daily life to **break down complex tasks** into **smaller, manageable steps**.
- **Why Use Layers?**
 - **Simplifies network design** by handling specific tasks at each layer.
 - **Enhances troubleshooting** since issues can be isolated to a single layer.
 - **Promotes compatibility** between different systems.
- **Example:** The **OSI model** and **TCP/IP model** use layering to structure network communication.

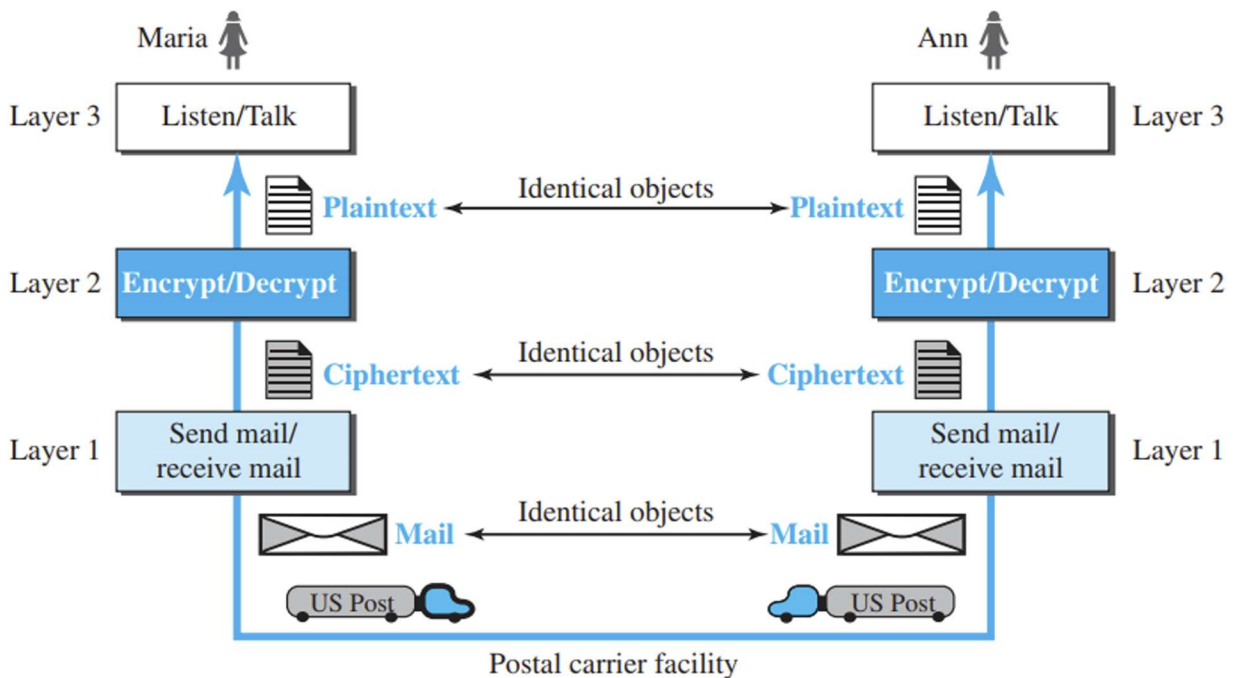


Fig: Tasks involved in sending a message in multiple layer

Importance of Layering

➔ Advantages of Protocol Layering:

1. Troubleshooting

- Each layer has specific **protocols, actions, and data**.
- Issues can be **pinpointed** to the layer responsible for the problem.

2. Standards & Interoperability

- Provides a **structured guideline** for different vendors.
- Ensures that **devices from different manufacturers** can communicate seamlessly.

3. Flexibility & Change Management

- **Changes in one layer** have **minimal impact** on others.
- Prevents disruptions across the entire system.

Introduction to TCP/IP Model

➔ TCP/IP (Internet Protocol Suite):

- **Definition:** A set of communication protocols used for the Internet and similar networks.
- **Key Protocols:** Named after **Transmission Control Protocol (TCP)** and **Internet Protocol (IP)**.
- **Function:** Enables data transmission and communication between networked devices.
- **Standardization:** One of the earliest networking protocol standards.
- **Usage:** Forms the foundation of modern internet communication.

TCP/IP Model in relation to OSI Model

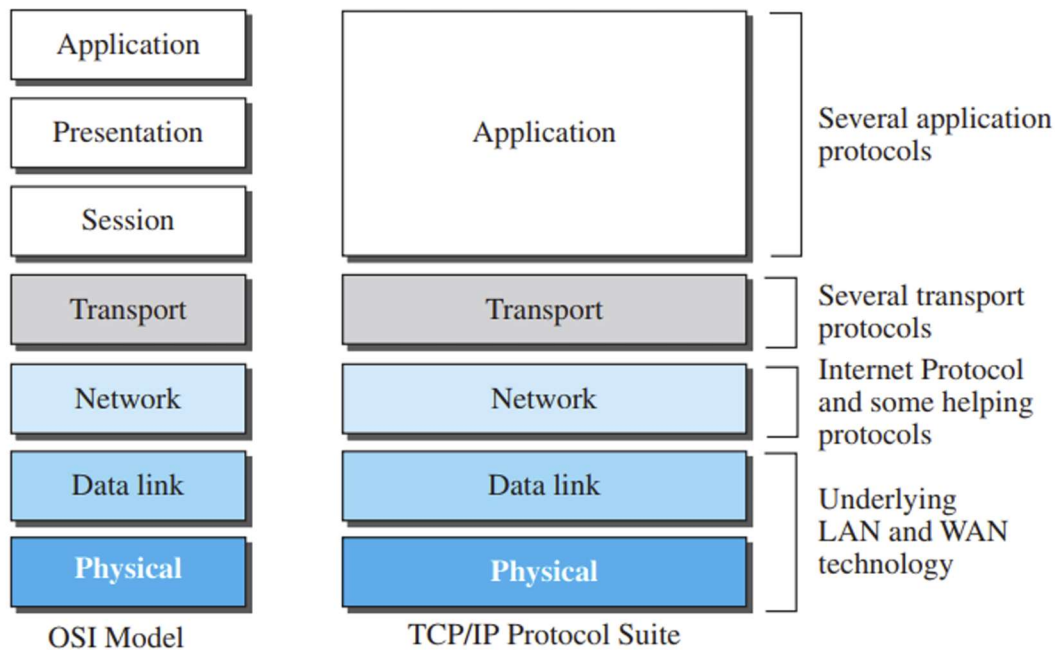


Fig: TCP/IP and OSI model

Application Layer

- **Function:** Only layer that directly interacts with user applications.
- **Role:**
 - Takes data from the user at the sender side.
 - Provides received data to the user at the receiver side.
- **Main Protocols:**
 - **HTTP (HyperText Transfer Protocol)** – for web browsing.
 - **DNS (Domain Name System)** – resolves domain names to IP addresses.
 - **FTP (File Transfer Protocol)** – transfers files between devices.
 - **SMTP (Simple Mail Transfer Protocol)** – used for sending emails.
- **Implementation:** Exists only in **end devices** (source and destination computers).

Example: When you open a webpage, your browser (application layer) uses HTTP to request content from the server.

Transport Layer

- Gets the message from the **application layer**, encapsulates it in a **segment** and sends it to **Network layer**.
- Delivery can be **reliable but slow** (TCP) or **unreliable but fast** (UDP).
- Provides **port addressing** for application programs.
- Handles **error control, flow control, and congestion control**.

Example: When you send an email, TCP ensures all parts of the message arrive correctly in order.

Network / Internet Layer

- Moves data **from source to destination** across multiple networks.
- **Encapsulates segments into packets.**
- **Finds the best path (routing)** for data delivery.
- Uses **logical addressing (IP address)** to identify devices.
- Handles **packet forwarding and fragmentation**.

Example: When sending data online, the IP address helps route packets to the correct device.

Data Link Layer

- Handles **communication within the same network**.
- Uses **MAC addresses** for device identification.
- Performs **error control** and **flow control**.
- **Encapsulates packets into frames.**
- Used in **PCs, routers, switches, bridges** (not in hubs/repeaters).
- **Protocols:** ALOHA, CSMA, CSMA/CD, CSMA/CA.

Example: When two devices on the same LAN communicate, they use their MAC addresses for frame transmission.

Physical Layer

- Takes **frames from Data-Link Layer** and converts them into **signals** for transmission.
- Defines the **physical transmission media** (cables, wireless, etc.).
- Deals with **network topology** (how devices are physically connected).

Example: An Ethernet cable carrying electrical signals represents the Physical Layer in action.

Data Encapsulation & Decapsulation

Encapsulation in the TCP/IP Model

1. **Application Layer:** Sends the email data to the Transport layer.
2. **Transport Layer:** Adds its own header (e.g., port information) and passes data to the Internet layer.
3. **Internet Layer:** Adds a header (e.g., source & destination IP addresses) and forwards data to the Network Access layer.
4. **Network Access Layer:** Adds both a header and a trailer before sending data through the physical network.

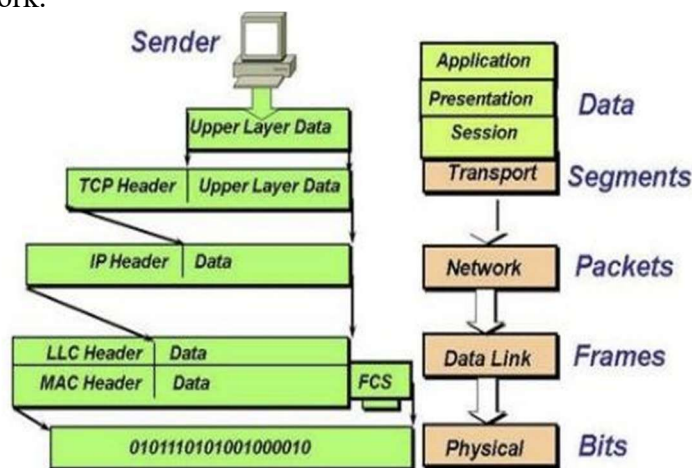


Figure: Data Encapsulation

➔ Decapsulation in the TCP/IP Model

- **Definition:** The reverse of encapsulation, where data is unpacked as it moves up the layers.
- **Process Flow:** Starts from the **Physical Layer** and moves up to the **Application Layer**.
- **Physical Layer:** Receives raw data and passes it to the **Network Access Layer**.
- **Network Access Layer:** Removes its header and trailer, then sends data to the **Internet Layer**.
- **Internet & Transport Layers:** Sequentially strip their headers and forward data upward.
- **Application Layer:** Receives the original data for processing by the intended program.

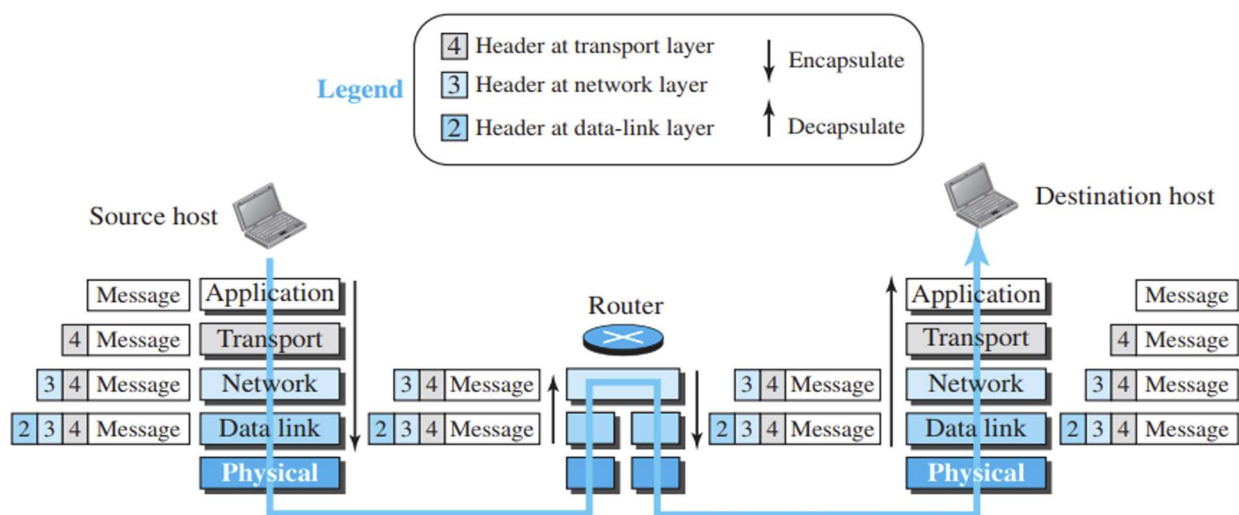


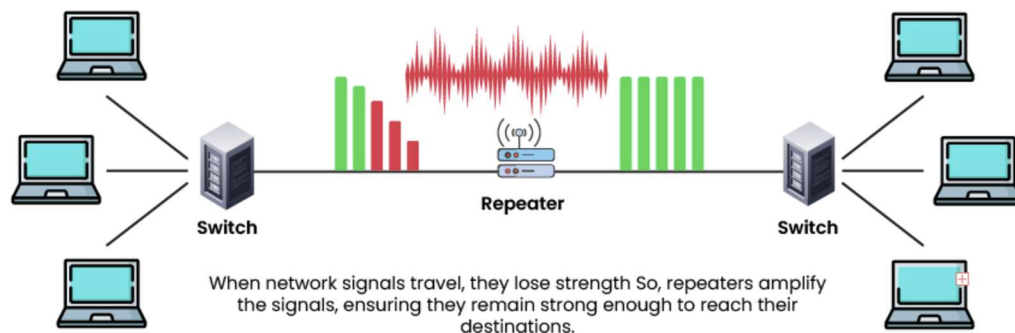
Figure: Encapsulation and decapsulation process

Connecting Devices

- ⇒ They are devices that link networked computers and direct data between them.
- ❖ **Repeater:** They are devices that boost signals to extend network range.
 - ❖ **Hub:** They are devices that connect multiple devices and broadcast data to all.
 - ❖ **Bridge:** They are devices that connect two network segments and filter data by MAC.
 - ❖ **Switch:** They are devices that send data only to the intended device using MAC.
 - ❖ **Router:** They are devices that connect different networks and route data using IP.

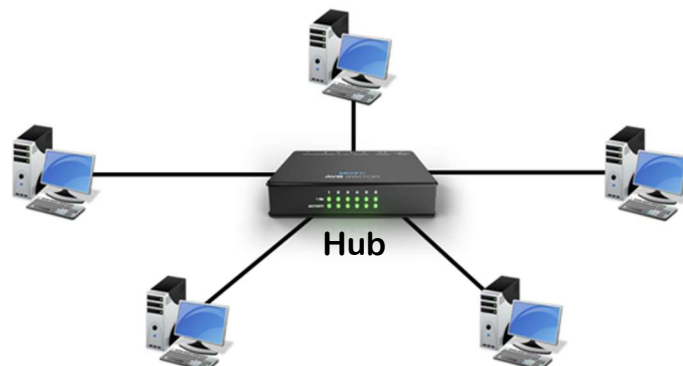
Repeater

- **Function:** Amplifies or regenerates weak signals to extend the transmission distance of a network.
- **Layer:** Physical Layer (**Layer 1** of OSI)
- **Use Case:** Extending LAN or Wi-Fi coverage without changing the data.
- **Key Point:** Does **not** filter data or address devices; just boosts signal.



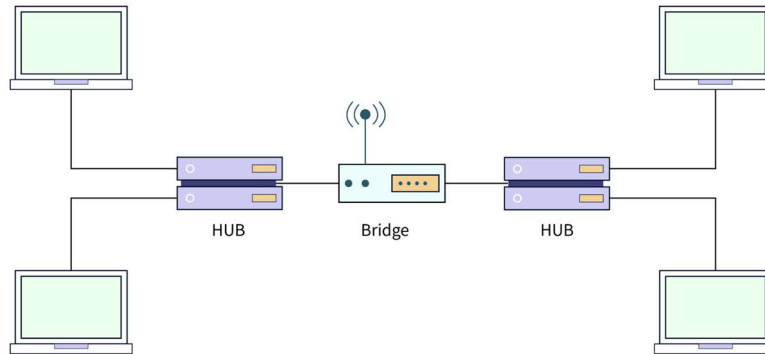
Hub

- **Function:** Connects multiple devices in a network and broadcasts incoming data to all connected devices.
- **Layer:** Physical Layer (**Layer 1**)
- **Use Case:** Small LAN where performance isn't critical.
- **Key Point:** **Inefficient**, because all devices receive all data, causing collisions.



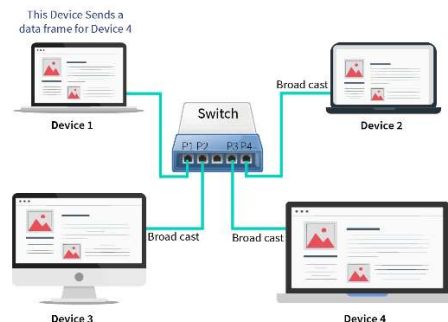
Bridge

- **Function:** Connects two LAN segments and filters traffic based on MAC addresses.
- **Layer:** Data Link Layer (**Layer 2**)
- **Use Case:** Separating network segments to reduce collisions.
- **Key Point:** Can **learn MAC addresses** and forward only relevant traffic. Predecessor of a switch.



Switch

- **Function:** Connects multiple devices and forwards data only to the destination device using MAC addresses.
- **Layer:** Data Link Layer (**Layer 2**), some advanced switches work at Layer 3.
- **Use Case:** Modern LANs for efficient traffic management.
- **Key Point:** Reduces collisions and improves bandwidth compared to hubs.



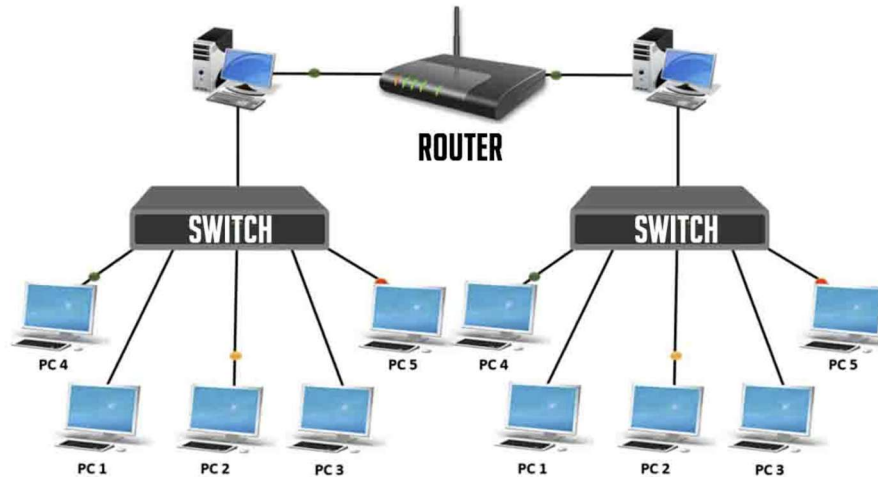
Steps :-

- 1 Device 1 send data frame for device 4 .
- 2 Since MAC table is empty .
- 3 Switch broadcast to all the device except 1.
- 4 Since the data frame was for device 4, it sends an acknowledgement.
- 5 Now switch maps post 4 to device 4 .

Post	MAC
1	-
2	-
3	-
4	MAC of device 4

Router

- **Function:** Connects multiple networks and routes data between them using IP addresses.
- **Layer:** Network Layer (**Layer 3**)
- **Use Case:** Connecting LANs to the Internet or different network segments.
- **Key Point:** Determines the **best path** for data, can provide NAT, DHCP, firewall features.



Quick Comparison Table

Device	OSI Layer	Traffic Forwarding	Use Case
Repeater	1	Amplifies all signals	Extend network distance
Hub	1	Broadcasts to all ports	Small networks
Bridge	2	Forwards based on MAC	Segment LANs
Switch	2	Forwards to specific MAC	Modern LANs
Router	3	Routes based on IP	Connect networks/Internet

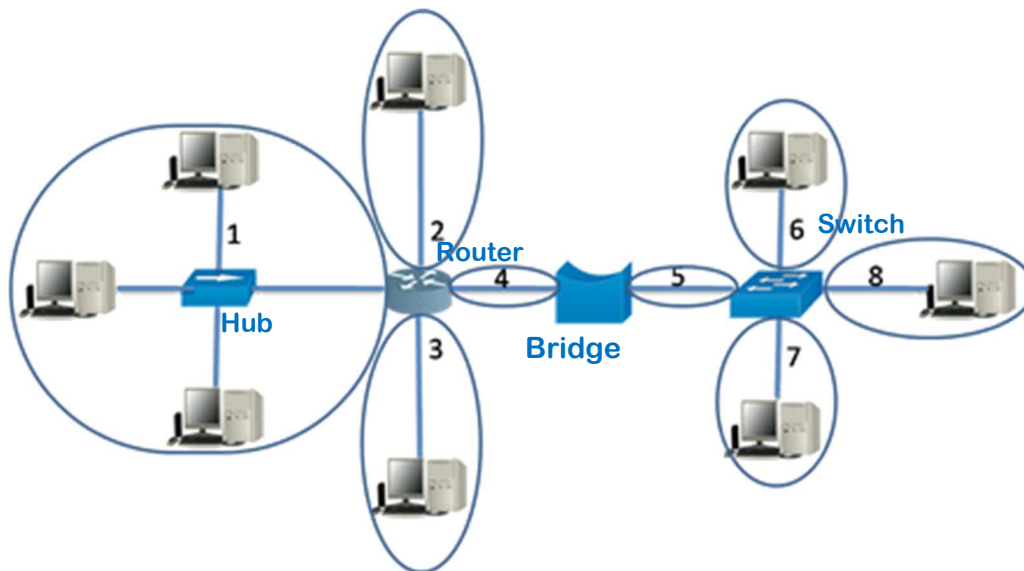
Collision Domain

⇒ A **collision domain** is a network segment where **two or more devices share the same network bandwidth** and **data packets can collide** if sent at the same time.

- Collisions occur in **half-duplex networks** like hubs and older Ethernet setups.
- When a collision happens, the devices must **retransmit**, which reduces network efficiency.

Key Points:

1. A **collision domain is limited by devices** that can separate traffic:
 - **Repeater** → does **not** separate collision domains.
 - **Hub** → all devices in a hub are in the **same collision domain**.
 - **Switch/ Bridge** → each port is a **separate collision domain**.
 - **Router** → separates collision domains completely.
2. **Full-duplex links** (like modern switches) **eliminate collisions**, so each port effectively has its own collision-free domain.



Broadcast Domain

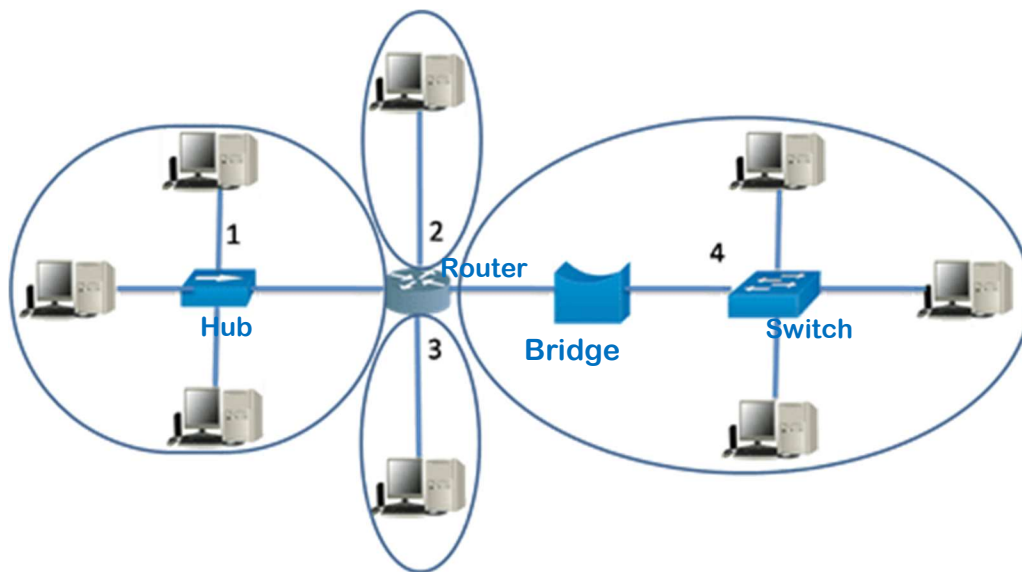
⇒ A **broadcast domain** is a network area where **all devices receive broadcast messages** sent by any device.

- Broadcasts are messages meant for **all devices** in the same segment.
- Devices outside this domain **won't receive the broadcast**.

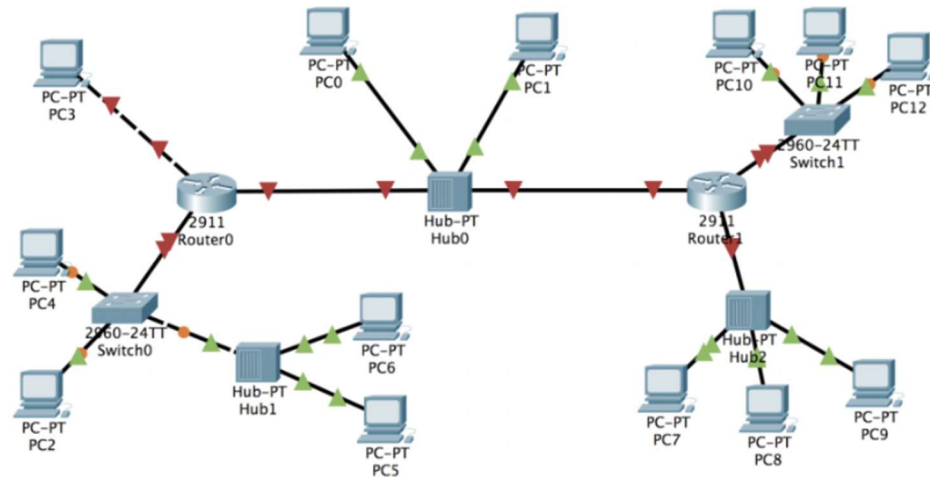
Key Points:

- **Hub** → does **not** separate broadcast domains.
- **Switch** → does **not** separate broadcast domains by default.
- **Router** → **separates broadcast domains**; broadcasts do not cross router interfaces.
- **Bridge** → typically does **not** separate broadcast domains; forwards broadcast frames.

Examples of broadcasts: ARP requests, DHCP requests.



Problem1: From Fig 1, find out the number of **Broadcast and Collision Domains** along with highlighting all the broadcast domains and collision domains using circle/ellipse



Solutions:

Broadcast Domains	Collision Domains
Number of Broadcast Domains: 5	Number of Collision Domains: 11

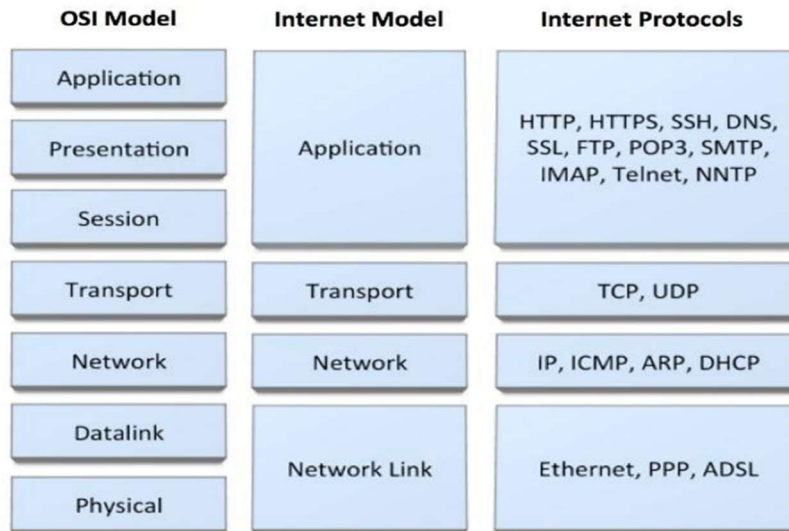
Wireless Communication

1. **DSSS:** Spreads data to reduce noise.
2. **FHSS:** Hops between frequencies to avoid interference.
3. **OFDM:** Sends data on many small frequencies at once.
4. **QPSK:** Sends data by changing signal phase.
5. **QAM:** Sends data by changing signal phase and strength.
6. **MIMO:** Uses many antennas for better speed and range.
7. **Beamforming:** Focuses signal toward one device.

APPLICATION LAYER PROTOCOLS

What is a Protocol?

- A **protocol** is a set of rules for data communication.
- It defines **what**, **how**, and **when** data is communicated.
- Example: **HTTP, FTP, TCP, IP**



Types of Protocols

1. Push Protocol:

- Server sends (pushes) new data to the client.
- Connection stays always on.
- Example: **SMTP**

2. Pull Protocol:

- Client requests (pulls) data from the server.
- Connection opens and closes after each request.
- Example: **HTTP**

HTTP (Hypertext Transfer Protocol)

- ⇒ Used on the **World Wide Web** for data transfer.
- ⇒ Works on **Request/Response** model.
- ⇒ Client = Browser; Server = Web server.

❖ HTTP Methods:

- **GET:** Request data from the server.
- **POST:** Send data (like form submission).
- **PUT:** Upload or update resources on the server.

❖ Runs on:

- **TCP Port 80**
- TCP ensures reliable delivery, so HTTP doesn't handle data loss.

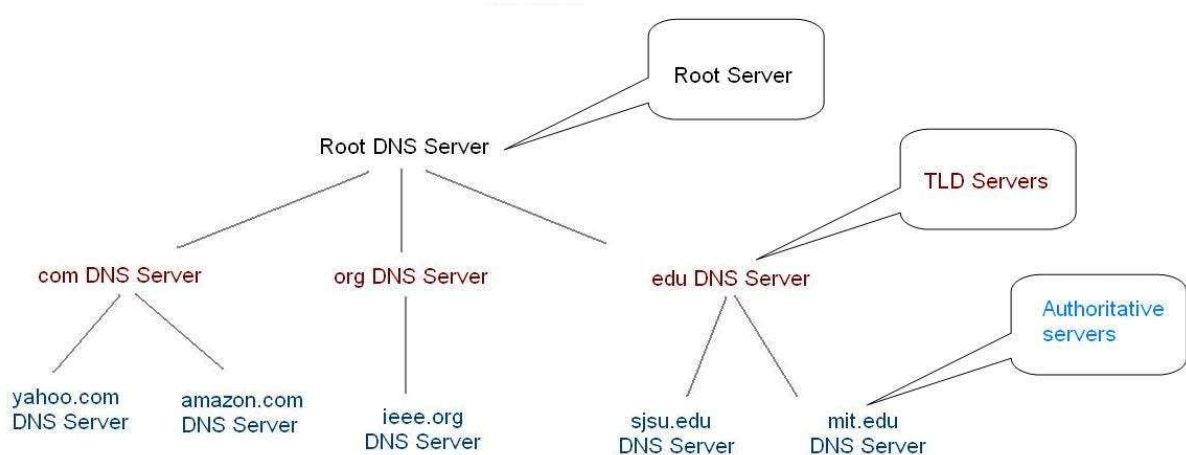
DNS (Domain Name System)

- ⇒ Translates **domain names** → **IP addresses**.
- ⇒ Works automatically to match names with addresses.
- ⇒ Uses many **Name Servers** around the world.
- ⇒ Example: **www.google.com** → **142.250.190.4**

❖ Types of DNS Servers:

1. **Root Server:** Knows top-level domains (.com, .edu, etc.)
2. **Local Server:** Belongs to ISPs or organizations; handles user queries.
3. **Authoritative Server:** Holds IP info for specific domains.

DNS Hierarchy



Internet Domain Names

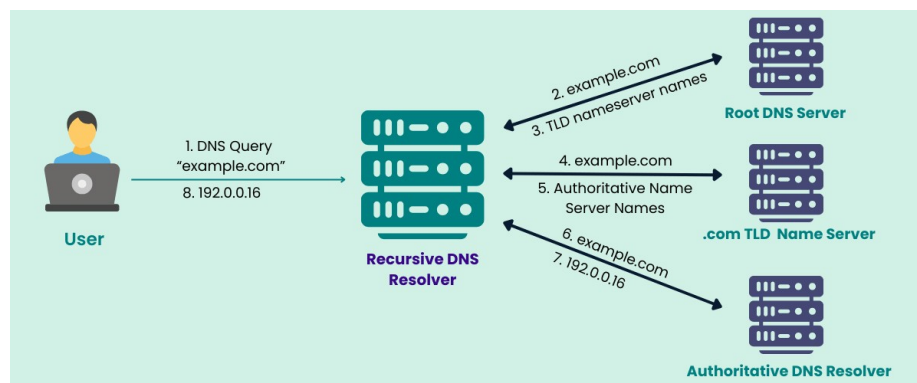
- ⇒ Format: **local.group.site**
- ⇒ Example: **cs.AIUB.edu**
 - **cs** → department (lowest level)
 - **AIUB** → university (second level)
 - **edu** → top-level domain

Common Domain Names and Their Meanings

Domain Name	Meaning
COM	Commercial organizations
EDU	Educational institutions (4-year)
GOV	Government institutions
MIL	Military groups
NET	Major network support centers
ORG	Organizations other than those above
ARPA	Temporary ARPANET domain (obsolete)
INT	International organizations
Country Code	Each country (geographic scheme)

How DNS Works

1. User types **www.aiub.com**
2. Local DNS checks its record.
3. If not found, it asks a **Root Server**.
4. Root Server refers it to **aiub.com** DNS.
5. That DNS replies with IP address → User connects.



“A useful link for better understand: <https://youtu.be/mpQZVYPuDGU?si=stmVmXi8yh0r8pag>”

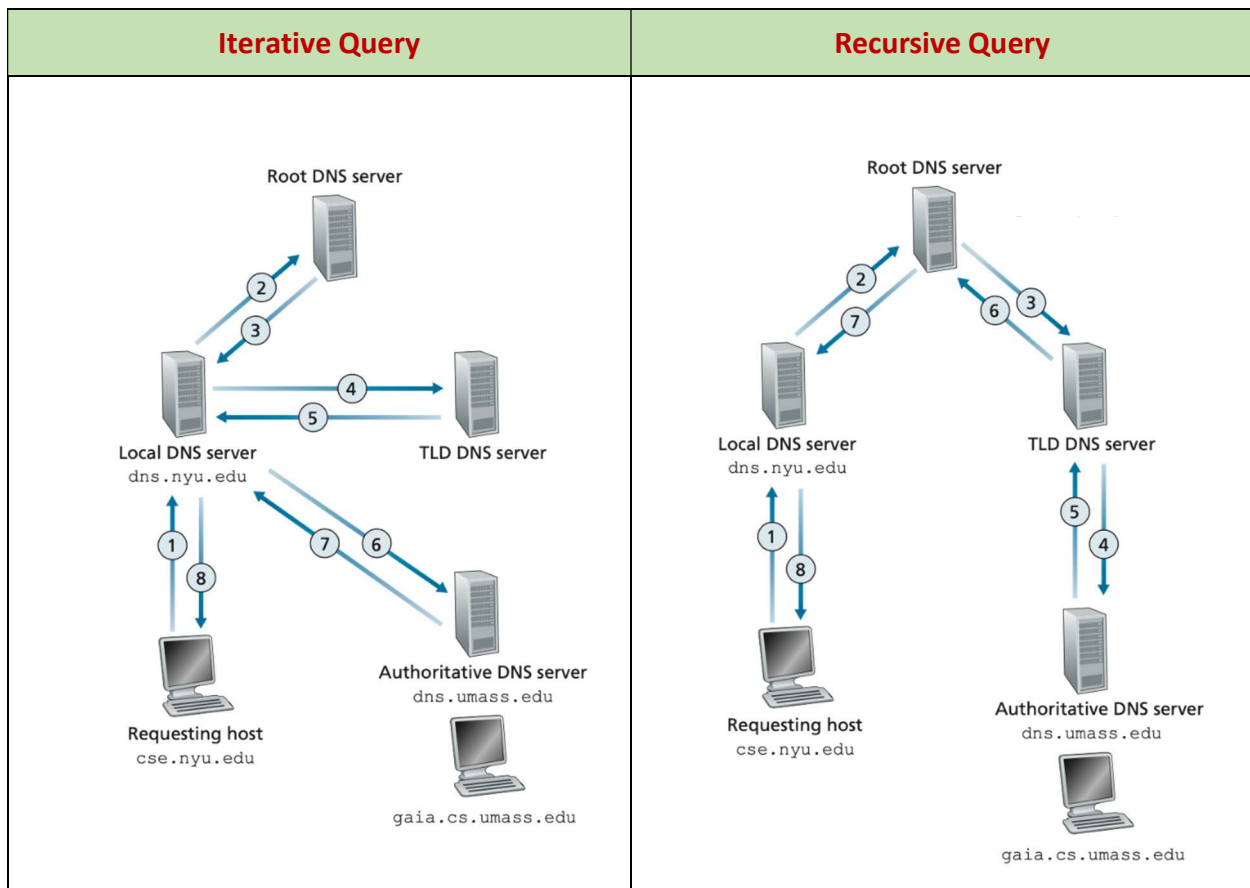
Types of DNS Queries

1. Recursive Query:

- DNS server finds the full answer for the client.
- May ask other servers on behalf of the client.

2. Iterative (Nonrecursive) Query:

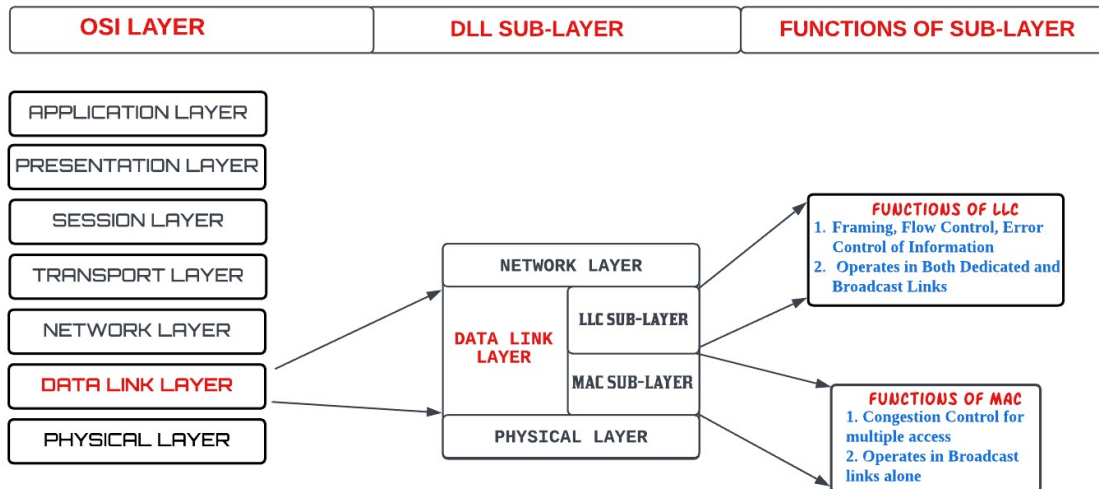
- DNS server returns a referral to another DNS server.
- The client then continues the search.



DATA LINK LAYER: MULTIPLE ACCESS TECHNIQUES 1

Sublayers of Data Link Layer

1. **LLC (Logical Link Control):** Handles **flow** and **error control**.
2. **MAC (Medium Access Control):** Handles **channel access** (who sends data and when).



What are Multiple Access Networks?

- ⇒ When **multiple devices share a common communication medium**.
- ⇒ Used in **LANs, cellular, and satellite networks**.

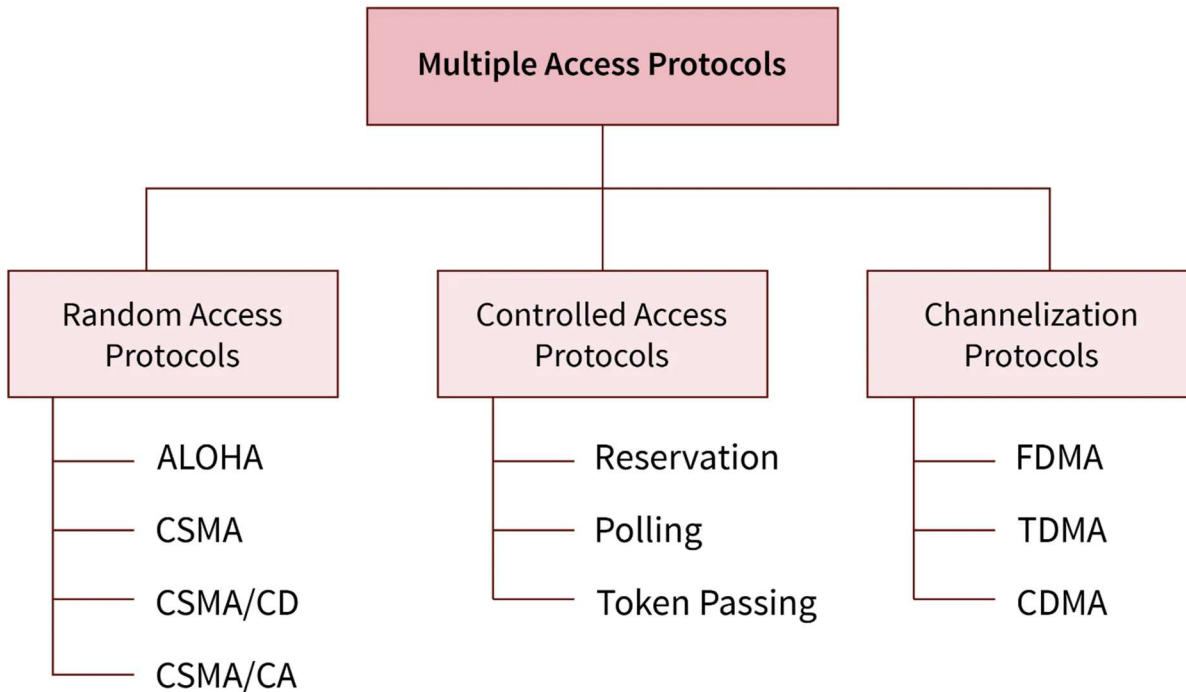
❖ Advantages:

- **Low cost:** Only one medium is shared by many stations.
- **No need for complex routing:** All stations can hear each other's transmissions.

❖ Disadvantages:

- **Must avoid collisions:** If many stations send data at once, **data collision** can occur.
- Need **coordination** so stations don't transmit data at the same time.
- **Fair access must be maintained:** One station shouldn't **occupy the medium all the time**.

Multiple Access Protocols



Channelization Protocols

⇒ Channelization is a multiple access method where Bandwidth is divided by *time*, *frequency*, or *code* among stations.

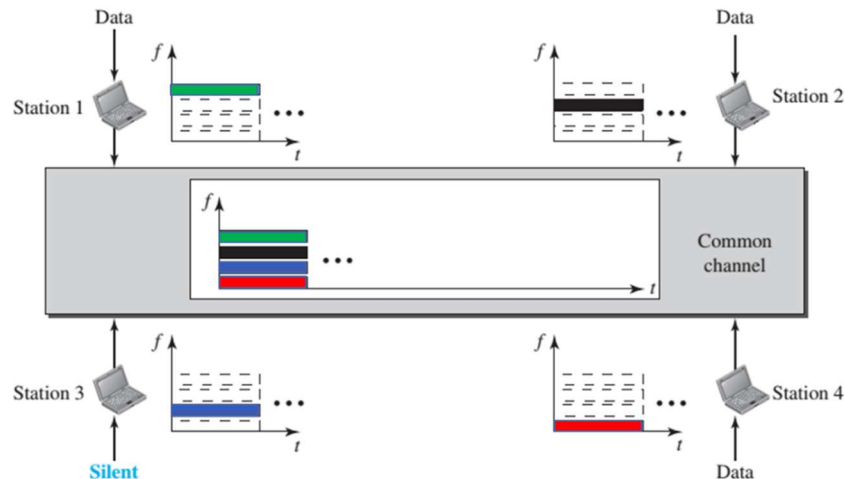
There are three basic channelization protocols:

1. **Frequency Division Multiple Access (FDMA),**
2. **Time Division Multiple Access (TDMA),**
3. **Code Division Multiple Access (CDMA).**

FDMA (Frequency Division Multiple Access)

- ⇒ The total bandwidth is divided into **different frequency bands**.
- ⇒ Each station gets its **own frequency band** all the time.
- ⇒ **Guard bands** (small empty spaces) are added to prevent overlap and interference.

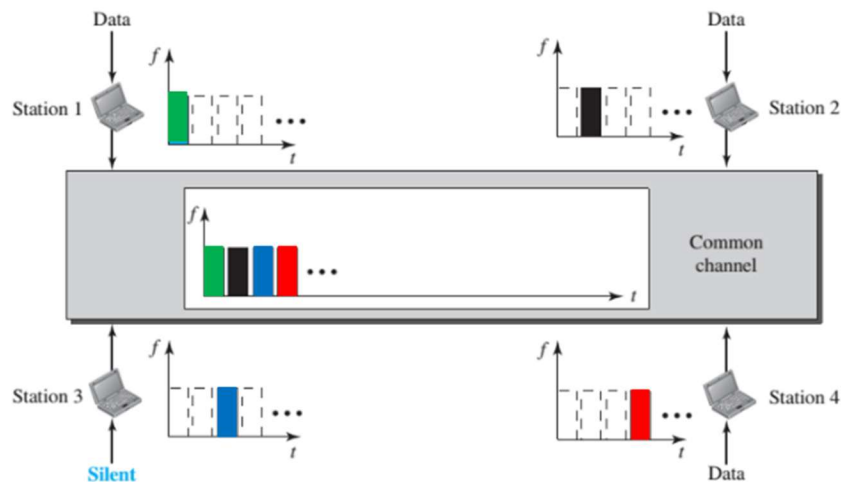
- **Example:** If there are 3 users, each gets a separate frequency to talk at the same time.
- **Used in:** Radio, Satellite, and Cellular systems.



TDMA (Time Division Multiple Access)

1. The whole channel is used by **each station one at a time**.
2. Each station gets a **time slot** to send data.
3. **Synchronization** is needed so each station knows when its time slot starts.

- **Example:** Like people taking turns to talk — one after another.
- **Used in:** GSM mobile systems.



CDMA (Code Division Multiple Access)

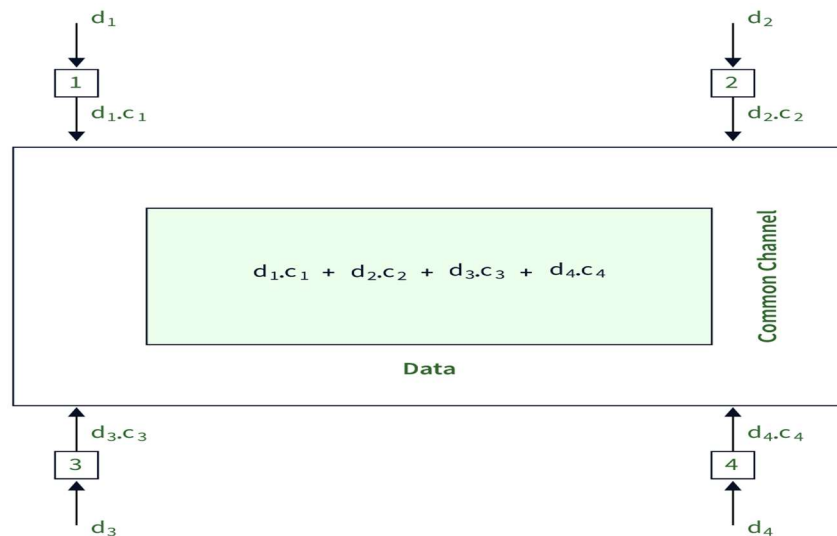
1. All users **transmit data at the same time** and use the **whole bandwidth**.
2. Each user has a **unique code** to identify their data.
3. The receiver uses that same code to extract the correct data.
4. Codes are **orthogonal** → multiplying two different codes gives **0** (no interference).

➤ **Properties of codes:**

Code $\times$ same code = number of stations

Code $\times$ different code = 0

- **Example:** It's like everyone speaking at once, but in **different languages**. You can understand your friend because you know their language (code).
- **Used in:** 3G cellular networks, Wi-Fi.



In the above diagram,

- Four stations: 1, 2, 3, 4.
- Each station has data: d_1, d_2, d_3, d_4 .
- Each station has a unique code: c_1, c_2, c_3, c_4 .
- Each station sends **data $\times$ code** (e.g., $d_1c_1, d_2c_2, \dots$)
- Channel carries the **sum**: $d_1c_1 + d_2c_2 + d_3c_3 + d_4c_4$.
- Receiver multiplies by the **sender's code** to get the data.
- Finally, the receiver divides by the **self-correlation value** to recover the original data.

✂ **Example:** Suppose there are 4 stations: 1, 2, 3, and 4. Each has its own code: c_1, c_2, c_3 , and c_4 . Each station sends data bits d_1, d_2, d_3 , and d_4 . Now, suppose station 2 wants to receive data from station 1.

Is given,

4 stations = 1, 2, 3, 4.

Code = c_1, c_2, c_3, c_4 .

data bits = d_1, d_2, d_3, d_4 .

So, the data on the channel will be:

$$data = (d_1c_1 + d_2c_2 + d_3c_3 + d_4c_4)$$

Now, since station 2 wants to receive data from station 1:

⇒ We must multiply the received data on the channel by c_1 (code of station 1):

$$\begin{aligned} data \times c_1 &= (d_1c_1 + d_2c_2 + d_3c_3 + d_4c_4) \times c_1 \\ &= d_1(c_1c_1) + d_2(c_2c_1) + d_3(c_3c_1) + d_4(c_4c_1) \\ &= d_1 \times 4 + d_2 \times 0 + d_3 \times 0 + d_4 \times 0 \\ &= d_1 \times 4 \end{aligned}$$

Now divide by 4:

$$data' = \frac{(d_1 \times 4)}{4} = d_1$$

Therefore, station 2 gets the data of station 1 correctly.

How to generate codes?

$$W_1 = [+1]$$

$$W_2 = \begin{bmatrix} W_1 & W_1 \\ W_1 & \bar{W}_1 \end{bmatrix} = \begin{bmatrix} +1 & +1 \\ +1 & +1 \end{bmatrix} = \begin{bmatrix} +1 & +1 \\ +1 & -1 \end{bmatrix}$$

$$W_4 = \begin{bmatrix} W_2 & W_2 \\ W_2 & \bar{W}_2 \end{bmatrix} = \begin{bmatrix} +1 & +1 & +1 & +1 \\ +1 & -1 & +1 & -1 \\ +1 & +1 & +1 & +1 \\ +1 & -1 & +1 & -1 \end{bmatrix} = \begin{bmatrix} +1 & +1 & +1 & +1 \\ +1 & -1 & +1 & -1 \\ +1 & +1 & -1 & -1 \\ +1 & -1 & -1 & +1 \end{bmatrix} = \begin{matrix} c_1 \\ c_2 \\ c_3 \\ c_4 \end{matrix}$$

$$W_8 = \begin{bmatrix} W_4 & W_4 \\ W_4 & \bar{W}_4 \end{bmatrix} = \begin{bmatrix} +1 & +1 & +1 & +1 & +1 & +1 & +1 & +1 \\ +1 & -1 & +1 & -1 & +1 & -1 & +1 & -1 \\ +1 & +1 & -1 & -1 & +1 & +1 & -1 & -1 \\ +1 & -1 & -1 & +1 & +1 & -1 & -1 & +1 \\ +1 & +1 & +1 & +1 & -1 & -1 & -1 & -1 \\ +1 & -1 & +1 & -1 & -1 & +1 & -1 & +1 \\ +1 & +1 & -1 & -1 & -1 & -1 & +1 & +1 \\ +1 & -1 & -1 & +1 & -1 & +1 & +1 & -1 \end{bmatrix} = \begin{matrix} c_1 \\ c_2 \\ c_3 \\ c_4 \\ c_5 \\ c_6 \\ c_7 \\ c_8 \end{matrix}$$

Note: Each row of W represents the code of a user.

✂ **Problem1:** Suppose there are **five users A, B, C, D, E**. They want to **send 0, +1, +1, -1, -1** bits, respectively. Generate the code for each user and find the transmit sequence. Suppose that a **user G wants to decode** the information sent **from the user B**. How can he do this?

Is given,

5 User = A, B, C, D, E.

We need Code = c_1, c_2, c_3, c_4, c_5 . [Since the number of codes is 5, use W_8]

data bits = $d_1 = 0, d_2 = +1, d_3 = +1, d_4 = -1, d_5 = -1$

So, the data on the channel will be:

$$\text{data} = (d_1c_1 + d_2c_2 + d_3c_3 + d_4c_4 + d_5c_5) \dots \dots \dots (i)$$

So, We know,

$$W_8 = \begin{bmatrix} W_4 & W_4 \\ W_4 & \bar{W}_4 \end{bmatrix} = \begin{bmatrix} +1 & +1 & +1 & +1 & +1 & +1 & +1 & +1 \\ +1 & -1 & +1 & -1 & +1 & -1 & +1 & -1 \\ +1 & +1 & -1 & -1 & +1 & +1 & -1 & -1 \\ +1 & -1 & -1 & +1 & +1 & -1 & -1 & +1 \\ +1 & +1 & +1 & +1 & -1 & -1 & -1 & -1 \\ +1 & -1 & +1 & -1 & -1 & +1 & -1 & +1 \\ +1 & +1 & -1 & -1 & -1 & -1 & +1 & +1 \\ +1 & -1 & -1 & +1 & -1 & +1 & +1 & -1 \end{bmatrix} \begin{matrix} = c_1 \\ = c_2 \\ = c_3 \\ = c_4 \\ = c_5 \\ = c_6 \\ = c_7 \\ = c_8 \end{matrix}$$

Now put the values in equation (i),

$$\text{data} = (d_1c_1 + d_2c_2 + d_3c_3 + d_4c_4 + d_5c_5)$$

$$\text{data} = \begin{matrix} 0 \times \\ +1 \times \\ +1 \times \\ -1 \times \\ -1 \times \end{matrix} \begin{bmatrix} +1 & +1 & +1 & +1 & +1 & +1 & +1 & +1 \\ +1 & -1 & +1 & -1 & +1 & -1 & +1 & -1 \\ +1 & +1 & -1 & -1 & +1 & +1 & -1 & -1 \\ +1 & -1 & -1 & +1 & +1 & -1 & -1 & +1 \\ +1 & +1 & +1 & +1 & -1 & -1 & -1 & -1 \end{bmatrix}$$

$$\text{data} = \begin{bmatrix} +0 & +0 & +0 & +0 & +0 & +0 & +0 & +0 \\ +1 & -1 & +1 & -1 & +1 & -1 & +1 & -1 \\ +1 & +1 & -1 & -1 & +1 & +1 & -1 & -1 \\ -1 & +1 & +1 & -1 & -1 & +1 & +1 & -1 \\ -1 & -1 & -1 & -1 & +1 & +1 & +1 & +1 \end{bmatrix}$$

$$\text{data} = [0 \quad 0 \quad 0 \quad -4 \quad 2 \quad 2 \quad 2 \quad -2]$$

$$\therefore \text{Transmit sequence data} = [0 \quad 0 \quad 0 \quad -4 \quad 2 \quad 2 \quad 2 \quad -2]$$

Now, since User G wants to receive data from user B:

⇒ We must multiply the received data on the channel by c_2 (code of User 2):

$$\text{Transmit data} \times c_2 = \begin{bmatrix} +0 & +0 & +0 & -4 & +2 & +2 & +2 & -2 \end{bmatrix} \times \begin{bmatrix} +1 & -1 & +1 & -1 & +1 & -1 & +1 & -1 \end{bmatrix}$$

$$data' = 0 + 0 + 0 + 4 + 2 - 2 + 2 + 2 = 8$$

Now divide by 8:

$$data' = \frac{8}{8} = +1$$

Therefore, User G gets the data of User B correctly

✂ **Problem2:** Suppose there are five users A, B, C, D. They want to send -3, +1, +5, -1 bits, respectively. Generate the code for each user and find the transmit sequence. Suppose that a user B wants to decode the information sent from the user A. How can he do this?

Is given,

4 User = A, B, C, D.

We need Code = c_1, c_2, c_3, c_4 . [Since the number of codes is 4, use W_4]

data bits = $d_1 = -3, d_2 = +1, d_3 = +5, d_4 = -1$

So, the data on the channel will be:

$$data = (d_1c_1 + d_2c_2 + d_3c_3 + d_4c_4) \dots \dots \dots (i)$$

So, We know,

$$W_4 = \begin{bmatrix} W_2 & W_2 \\ W_2 & \bar{W}_2 \end{bmatrix} = \begin{bmatrix} \begin{matrix} +1 & +1 \\ +1 & -1 \end{matrix} & \begin{matrix} +1 & +1 \\ +1 & -1 \end{matrix} \\ \begin{matrix} +1 & +1 \\ +1 & -1 \end{matrix} & \begin{matrix} -1 & -1 \\ -1 & +1 \end{matrix} \end{bmatrix} = \begin{matrix} c_1 \\ c_2 \\ c_3 \\ c_4 \end{matrix}$$

Now put the values in equation (i),

$$data = (d_1c_1 + d_2c_2 + d_3c_3 + d_4c_4)$$

$$data = \begin{matrix} -3 \times \\ +1 \times \\ +5 \times \\ -1 \times \end{matrix} \begin{bmatrix} +1 & +1 & +1 & +1 \\ +1 & -1 & +1 & -1 \\ +1 & +1 & -1 & -1 \\ +1 & -1 & -1 & +1 \end{bmatrix}$$

$$data = \begin{bmatrix} -3 & -3 & -3 & -3 \\ +1 & -1 & +1 & -1 \\ +5 & +5 & -5 & -5 \\ -1 & +1 & +1 & -1 \end{bmatrix}$$

$$data = [+2 \quad +2 \quad -6 \quad -10]$$

$$\therefore \text{Transmit sequence data} = [+2 \quad +2 \quad -6 \quad -10]$$

Now, since User B wants to receive data from user A:

⇒ We must multiply the received data on the channel by c_1 (code of User 1):

$$\text{Transmit data} \times c_1 = [+2 \quad +2 \quad -6 \quad -10] \times \begin{bmatrix} +1 & +1 & +1 & +1 \end{bmatrix}$$

$$data' = 2 + 2 - 6 - 10 = -12$$

Now divide by 8:

$$data' = \frac{-12}{4} = -3$$

Therefore, User B gets the data of User A correctly

✂ **Problem3:** Suppose there are **five users A, B, C, D E**. They want to **send -1, +3, 0, +1, +2** bits, respectively. Generate the code for each user and find the transmit sequence. Suppose that a **user G wants to decode** the information sent **from the user B**. How can he do this?

Is given,

5 User = A, B, C, D, E.

We need Code = c_1, c_2, c_3, c_4, c_5 . [Since the number of codes is 5, use W_8]

data bits = $d_1 = -1, d_2 = +3, d_3 = 0, d_4 = +1, d_5 = +2$

So, the data on the channel will be:

$$\text{data} = (d_1c_1 + d_2c_2 + d_3c_3 + d_4c_4 + d_5c_5) \dots \dots \dots (i)$$

So, We know,

$$W_8 = \begin{bmatrix} W_4 & W_4 \\ W_4 & \bar{W}_4 \end{bmatrix} = \begin{bmatrix} +1 & +1 & +1 & +1 & +1 & +1 & +1 & +1 \\ +1 & -1 & +1 & -1 & +1 & -1 & +1 & -1 \\ +1 & +1 & -1 & -1 & +1 & +1 & -1 & -1 \\ +1 & -1 & -1 & +1 & +1 & -1 & -1 & +1 \\ +1 & +1 & +1 & +1 & -1 & -1 & -1 & -1 \\ +1 & -1 & +1 & -1 & -1 & +1 & -1 & +1 \\ +1 & +1 & -1 & -1 & -1 & -1 & +1 & +1 \\ +1 & -1 & -1 & +1 & -1 & +1 & +1 & -1 \end{bmatrix} \begin{matrix} = c_1 \\ = c_2 \\ = c_3 \\ = c_4 \\ = c_5 \\ = c_6 \\ = c_7 \\ = c_8 \end{matrix}$$

Now put the values in equation (i),

$$\text{data} = (d_1c_1 + d_2c_2 + d_3c_3 + d_4c_4 + d_5c_5)$$

$$\text{data} = \begin{matrix} -1 \times \\ +3 \times \\ +0 \times \\ +1 \times \\ +2 \times \end{matrix} \begin{bmatrix} +1 & +1 & +1 & +1 & +1 & +1 & +1 & +1 \\ +1 & -1 & +1 & -1 & +1 & -1 & +1 & -1 \\ +1 & +1 & -1 & -1 & +1 & +1 & -1 & -1 \\ +1 & -1 & -1 & +1 & +1 & -1 & -1 & +1 \\ +1 & +1 & +1 & +1 & -1 & -1 & -1 & -1 \end{bmatrix}$$

$$\text{data} = \begin{bmatrix} -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 \\ +3 & -3 & +3 & -3 & +3 & -3 & +3 & -3 \\ +0 & +0 & -0 & -0 & +0 & +0 & -0 & -0 \\ +1 & -1 & -1 & +1 & +1 & -1 & -1 & +1 \\ +2 & +2 & +2 & +2 & -2 & -2 & -2 & -2 \end{bmatrix}$$

$$\text{data} = [+5 \quad -3 \quad +3 \quad -1 \quad +1 \quad -7 \quad -1 \quad -5]$$

$$\therefore \text{Transmit sequence data} = [+5 \quad -3 \quad +3 \quad -1 \quad +1 \quad -7 \quad -1 \quad -5]$$

Now, since User G wants to receive data from user B:

⇒ We must multiply the received data on the channel by c_2 (code of User 2):

$$\text{Transmit data} \times c_2 = \begin{bmatrix} +5 & -3 & +3 & -1 & +1 & -7 & -1 & -5 \end{bmatrix} \times \begin{bmatrix} +1 & -1 & +1 & -1 & +1 & -1 & +1 & -1 \end{bmatrix}$$

$$data' = 5 + 3 + 3 + 1 + 1 + 7 - 1 + 5 = 24$$

Now divide by 8:

$$data' = \frac{24}{8} = +3$$

Therefore, User G gets the data of User B correctly

Difference Between FDMA, TDMA, CDMA

Feature	FDMA	TDMA	CDMA
Basic idea	Each user gets a separate frequency band	Each user gets a separate time slot	All users share the same band but have unique codes
Channel separation	By frequency	By time	By code
Simultaneous users	Transmit at the same time , different frequencies	Transmit at different times , same frequency	Transmit simultaneously using different codes
Synchronization need	Low	High (precise timing needed)	Medium (code synchronization)
Interference	Adjacent channel interference	Time slot overlap interference	Multiple access interference (if codes not perfectly orthogonal)
Bandwidth efficiency	Moderate	High	Very high
Example use	Analog cellular (1G)	GSM (2G)	3G mobile, GPS, modern wireless systems

Controlled Access Protocols

- ⇒ **Controlled Access Protocols** are network protocols where **stations coordinate among themselves** to determine which one can transmit data at a given time — avoiding collisions by **controlling access** to the shared communication channel.

There are three popular Controlled Access protocols:

1. **Reservation**
2. **Polling**
3. **Token Passing**

Reservation

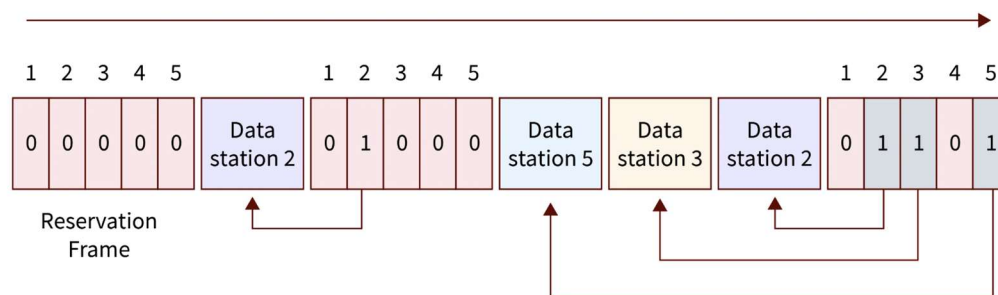
1. Each station **reserves** the channel before data transmission.
2. The time is divided into **slots**; a station sends a **reservation message** to claim a slot.
3. Common in systems where predictable access is needed (e.g., satellite networks).

How it works:

- If there are **N stations** in the system, there are exactly **N reservation minislots** in the reservation frame.
- Each minislots belongs to a station.
- When a station needs to send a data frame, it makes a reservation in its own minislots.
- The stations that have made reservations can send their data frame after the reservation frame.

➤ **Example:** Like booking a seat before watching a movie — once reserved, you get your turn.

Reservation access method:



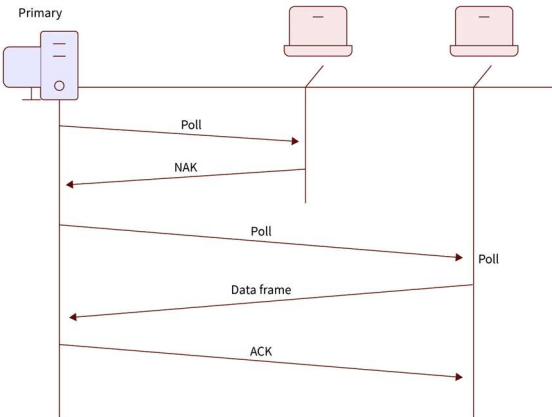
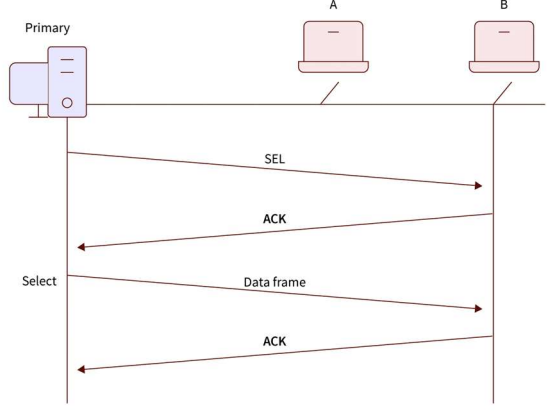
Let us take an example, through which we learn how reservations work. Suppose there are 5 stations and also 5 mini slot reservation frames. If in the first interval, only the 2nd, 3rd, and 5th stations are allowed to make the reservation and then after that in the second interval only the second station is allowed to make the reservation.

Polling

1. A **primary station (controller)** controls communication.
2. **Secondary stations** can only send/receive when allowed by **primary station**.
3. Two operations:
 - **Poll function:** Primary asks if secondary has data to send.
 - **Select function:** Primary tells secondary to receive data.

➤ **Example:** Like a teacher asking each student one by one to speak.

➤ **Used in:** master–slave or client–server setups.

Poll function	Select function
 <pre> sequenceDiagram participant Primary participant A participant B Primary->>A: Poll A->>Primary: NAK Primary->>B: Poll B->>Primary: Data frame Primary->>B: ACK </pre>	 <pre> sequenceDiagram participant Primary participant A participant B Primary->>A: SEL A->>Primary: ACK Primary->>B: Select B->>Primary: Data frame Primary->>B: ACK </pre>
• Primary wants to receive data from secondary. • It polls each secondary one by one. • If no data → secondary sends NAK. • Primary then polls next device. • If data available → secondary sends data frame. • Primary receives it and sends ACK.	• Primary wants to send data to secondary. • Sends a Select (SEL) frame first. • SEL frame has the secondary's address. • Alerts secondary to get ready for data. • Secondary sends ACK back. • Primary then sends the data frame.

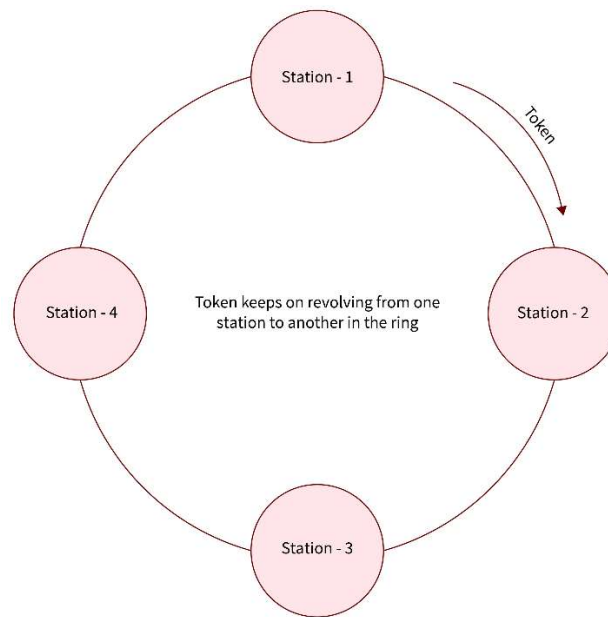
Token Passing

1. Stations are arranged in a **logical ring** (a circular connection).
2. A **token (small control frame)** circulates among stations.
3. Only the station holding the token can send data.
4. After transmission data, it **passes the token** to the next station in order.

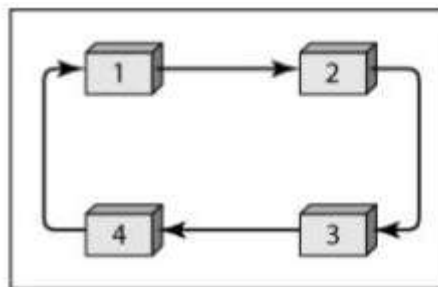
➤ **Example:** Like passing a “talking stick” — only the one holding it can speak.

➤ **Used in:** Networks like **Token Ring** or **FDDI**.

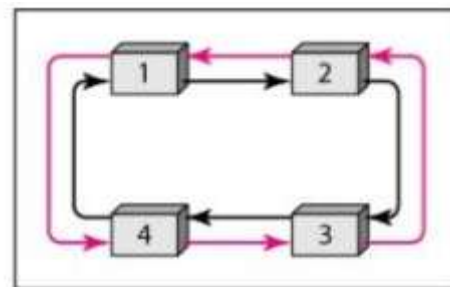
Token Passing: Using Physical ring



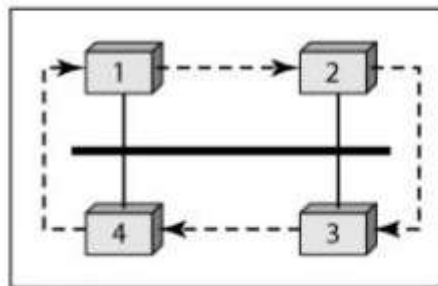
Token Passing: Using Logical ring and physical topology



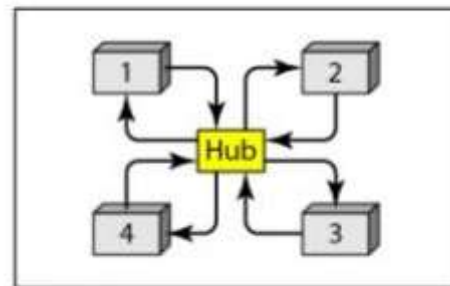
a. Physical ring



b. Dual ring



c. Bus ring



d. Star ring

DATA LINK LAYER: MULTIPLE ACCESS TECHNIQUES 2

Random Access Protocols

- ⇒ Stations transmit **whenever they have data**, without prior coordination.
- ⇒ **Collisions** may occur when two or more stations transmit at the same time.
- ⇒ If a collision happens, stations **retry after a random time**.

There are four basic Random Access protocols:

1. **ALOHA – Pure and Slotted**
2. **CSMA – Carrier Sense Multiple Access**
3. **CSMA/CD – CSMA with Collision Detection**
4. **CSMA/CA – CSMA with Collision Avoidance**

ALOHA

- ⇒ ALOHA is a **random-access protocol** in which stations transmit data **whenever they have data to send**, without checking if the channel is free. Collisions may occur, and if a collision happens, the stations **retransmit after a random time**.

➤ Types of ALOHA in Computer Networks

1. Pure ALOHA

- Stations **transmit anytime** without checking the channel.
- Collisions may occur; collided frames are **retransmitted after a random time**.
- **Efficiency:** ~18%.

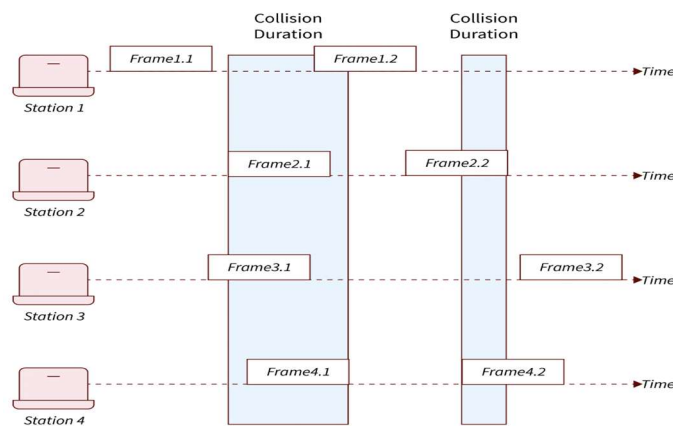
2. Slotted ALOHA

- Time is divided into **slots**; stations transmit only at the **start of a slot**.
- Reduces collisions compared to Pure ALOHA.
- **Efficiency:** ~37%.

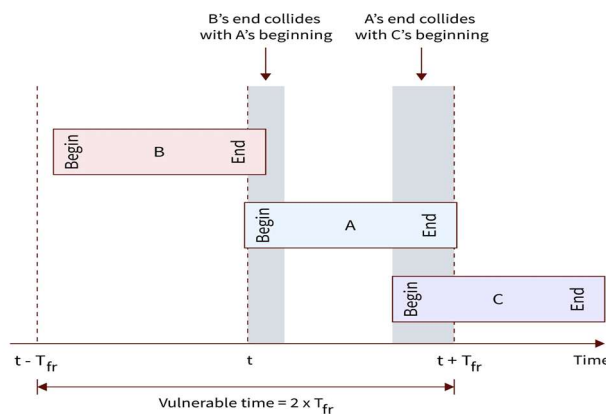
Pure ALOHA

- Each station sends data **whenever it wants**.
- If two stations send data at the same time → **collision** happens.
- Sender waits for **acknowledgment (ACK)** from the receiver.
- If no ACK is received (because of collision), it waits for a **random backoff time** before resending.
- After several failed attempts (Kmax), it stops trying for a while.
- **Vulnerable time** = $2 \times T_{fr}$ (T_{fr} = time to send one frame).
- **Efficiency**: Low (about 18%).

Collisions in Shared Channel:



Vulnerable Time for Pure Aloha:



$$Vulnerable\ time = 2 \times T_{fr}$$

Throughput of Pure Aloha:

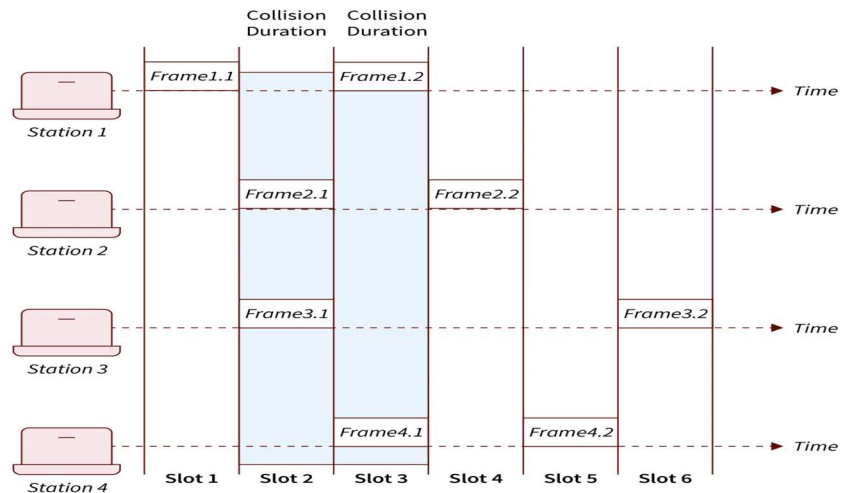
$$S = G * e^{-2G}$$

G = average number of frames created by the system during a single-frame transmission period.

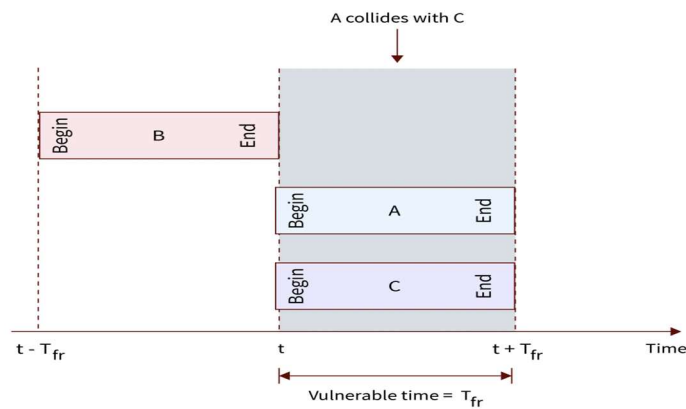
Slotted ALOHA

- Time is divided into **slots** (each slot = frame transmission time).
- Stations can only send **at the start** of a slot.
- Reduces collision chances.
- **Vulnerable time = T_{fr}** .
- **Efficiency:** Higher than Pure ALOHA (about 36%).

➤ Collisions in Shared Channel:



➤ Vulnerable Time for Pure Aloha:



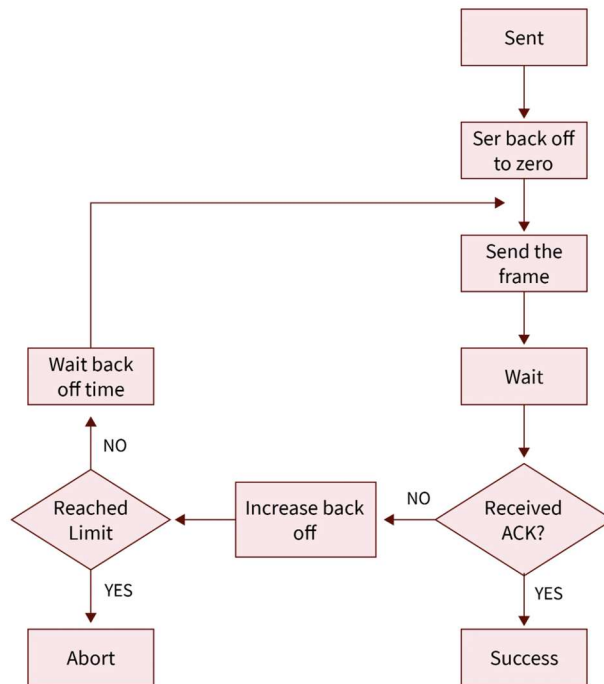
$$\text{Vulnerable time} = T_{fr}$$

➤ Throughput of Pure Aloha:

$$S = G * e^{-G}$$

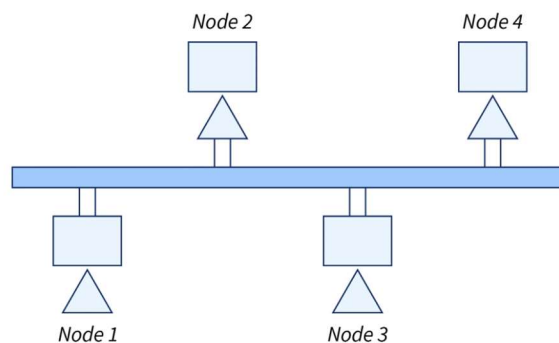
G = average number of frames created by the system during a single-frame transmission period.

Protocol Flow Chart for ALOHA



CSMA (Carrier Sense Multiple Access)

- ⇒ **Idea:** Before sending, each station **listens (senses)** the channel to check if it's free. This helps reduce collisions compared to ALOHA.

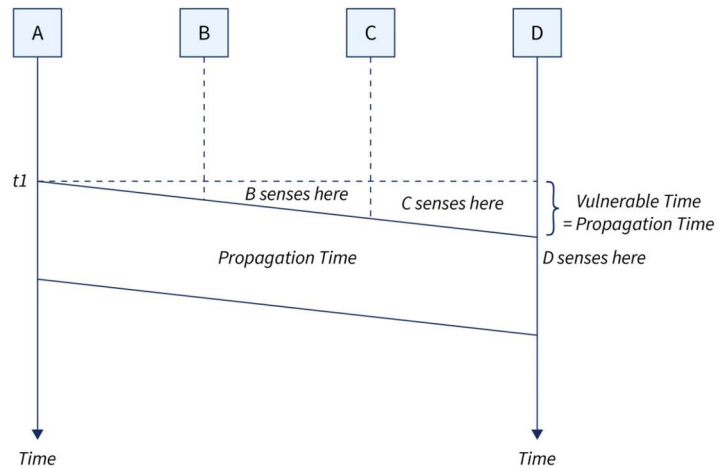


➤ Why collisions still occur:

- ⇒ Because of **propagation delay** – one station might sense the channel idle while another is already transmitting (signal hasn't reached it yet).

➤ Vulnerable Time in CSMA:

- **Vulnerable time = Propagation time (T_p).**
- T_p : time for a frame's first bit to travel from one end of the channel to the other.
- If **two stations transmit simultaneously**, a **collision** occurs.
- Once the **first bit reaches the end**, all stations **sense the channel as busy** and **stop transmitting**.

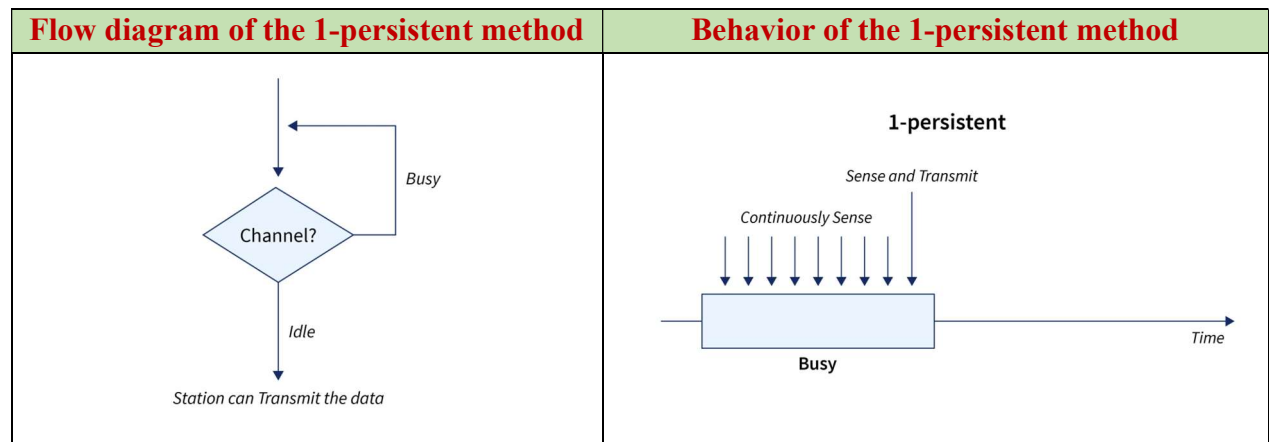


➤ Types of CSMA:

1. **1-Persistent CSMA**
2. **Non-Persistent CSMA**
3. **P-Persistent CSMA**

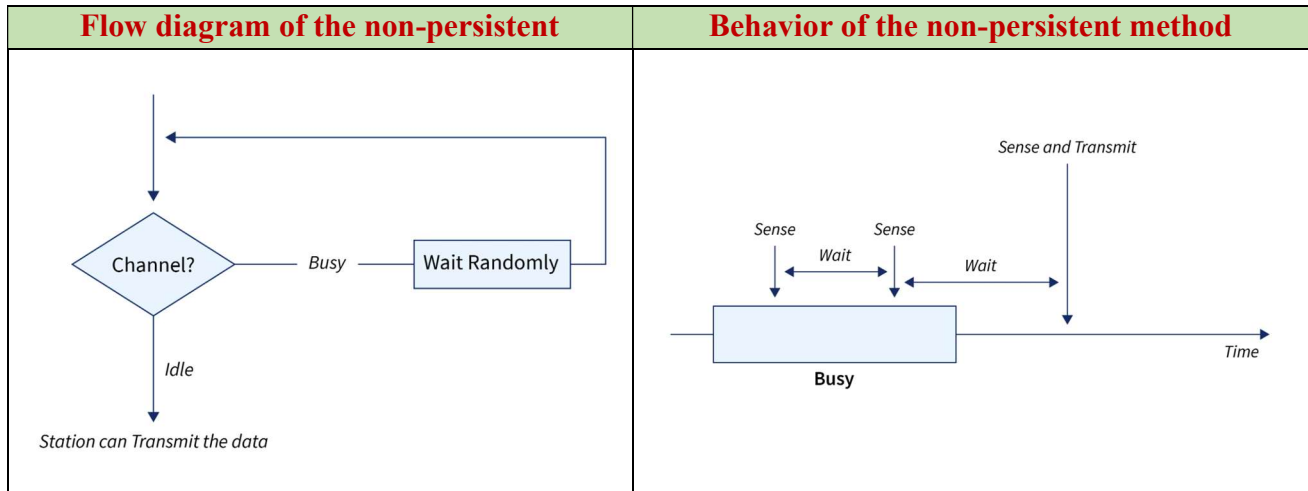
1-Persistent CSMA

- ⇒ Sense the channel **continuously**
- ⇒ if channel is **idle**, send immediately (probability = 1).
- ⇒ If channel is **busy**, keep sensing until it's free.
- ⇒ **Problem:** High chance of collision since many station may send instantly.



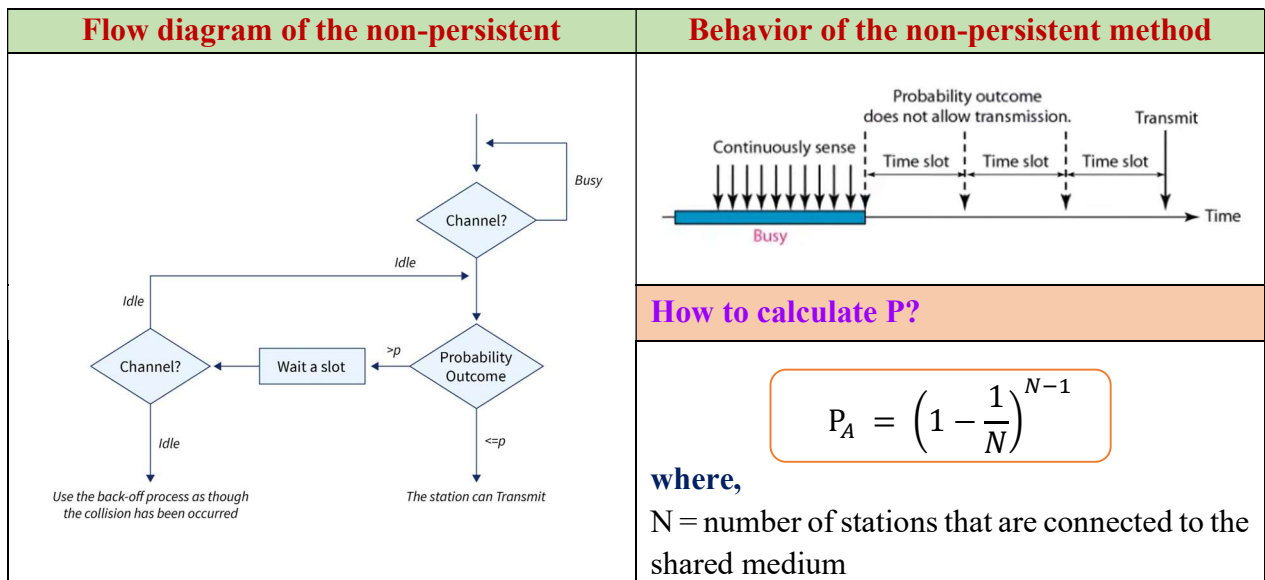
Non-Persistent CSMA

- ⇒ If channel is **busy**, wait a **random time** before checking again.
- ⇒ Advantage: Fewer collisions.
- ⇒ **Disadvantage**: Lower efficiency (channel may stay idle unnecessarily).



P-Persistent CSMA

- ⇒ Used in **slotted channels**.
- ⇒ If channel is **idle**:
 - Send with probability **p**.
 - Wait for next slot with probability **1 - p**.
- ⇒ Combines the benefits of the above two methods.



CSMA/CD (CSMA with Collision Detection)

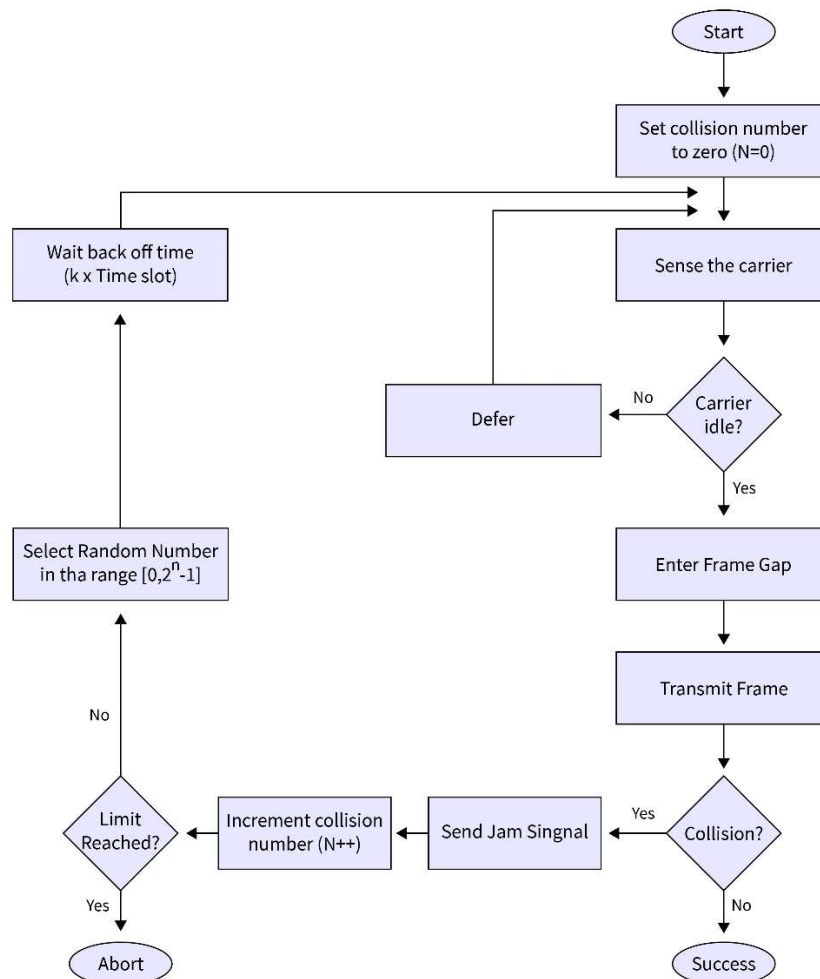
Full form: Carrier Sense Multiple Access with Collision Detection

Used in: Wired LANs (like Ethernet)

➤ How its Working:

- ⇒ Station checks if channel is idle.
- ⇒ Starts sending data.
- ⇒ **Continues to listen** while transmitting.
- ⇒ If collision is detected → transmission stops.
- ⇒ Sends a **jam signal** to alert others.
- ⇒ Waits for a random backoff time before retrying.

CCSMA/CD Procedure



CSMA/CA (CSMA with Collision Avoidance)

- ⇒ A **network protocol** used to **avoid collisions** during data transmission (mainly in **wireless networks**).
- ⇒ Station sends a **data frame** and waits for an **acknowledgment (ACK)**.
- ⇒ If it receives **only its signal**, the channel is **idle** → successful transmission.
- ⇒ If it detects **multiple signals**, a **collision** has occurred.
- ⇒ **CSMA/CA avoids collisions** using three techniques:
 1. **Interframe Space (IFS)**: short waiting time before sending data.
 2. **Contention Window**: random backoff time before retrying.
 3. **Acknowledgment (ACK)**: confirms successful data reception.

Full form: Carrier Sense Multiple Access with Collision Detection

Used in: Wired LANs (like Ethernet)

➤ **Why CSMA/CD doesn't work in wireless:**

- Wireless stations can't send and listen at the same time (half-duplex).
- Signal fading or obstacles may hide other transmissions.

Interframe Space (IFS)

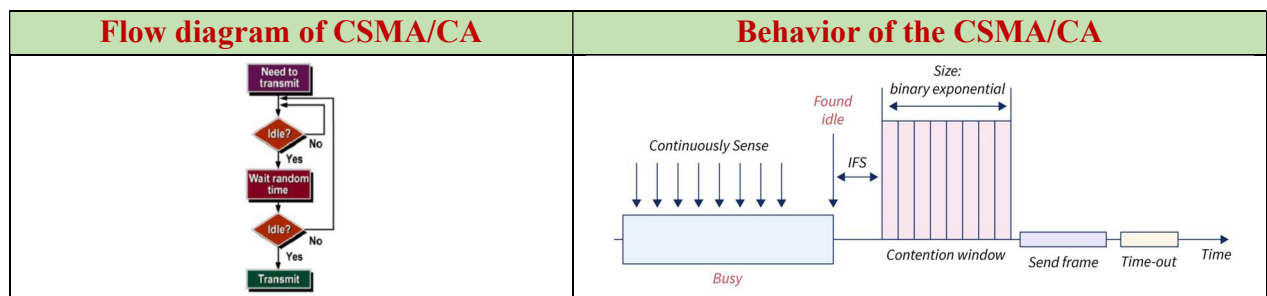
- ✓ After finding the channel idle, the station **waits for a short time** before sending.
- ✓ Purpose: To ensure no distant station has already started sending.

Contention Window

- ✓ Time is divided into small **slots**.
- ✓ Each station picks a **random number of slots** to wait.
- ✓ After waiting, it senses the channel again.
- ✓ If still busy, the window size doubles.

Acknowledgement

- ✓ Receiver sends an **ACK** after successful reception.
- ✓ If no ACK → sender assumes collision and retransmits after timeout.



ROUTING

Routing

- ⇒ Routing is the process of forwarding network packets from a **source network** to a **destination network** using a **router's routing table**.

A simplified routing table:

Destination Network	Subnet Mask	Next Hop	Interface	Type / Protocol	Metric/AD
192.168.1.0	255.255.255.0	192.168.2.1	eth0	Static	1/1
10.0.0.0	255.0.0.0	192.168.3.1	eth1	Static	2/1
172.16.0.0	255.240.0.0	10.0.0.2	eth2	OSPF	15/110
192.168.4.0	255.255.255.0	10.0.0.3	eth2	RIP	20/120
0.0.0.0 (Default)	0.0.0.0	Directly connected	eth0	-	5/0

- ⇒ **Destination Network:** The network you want to reach.
- ⇒ **Subnet Mask:** Defines the size of the network.
- ⇒ **Next Hop / Gateway:** The IP address of the router to forward the packet to.
- ⇒ **Interface:** The local network interface used (e.g., eth0, eth1).
- ⇒ **Metric:** Indicates route preference — **lower = better**.

Routing Table Creation

1. Static Routing

- Manually added by a network engineer.
- Best for **small networks**.
- Doesn't change automatically.

2. Dynamic Routing

- Automatically created and updated by **routing protocols**.
- Best for **medium to large networks**.
- Routers exchange information and learn about new paths.

Dynamic Routing Features

- ⇒ Shares routing info between routers.
- ⇒ Automatically learns and updates routes.
- ⇒ Finds the **best path** to each destination.

Types of Dynamic Routing Protocols

Based on Area:

Classification	Example
IGP (Interior Gateway Protocol) – used within an organization	RIP, IGRP, EIGRP, OSPF, IS-IS
EGP (Exterior Gateway Protocol) – used between organizations	BGP

Based on Algorithm:

Distance Vector Protocols	Link State Protocols
• Share routing tables with neighbors. • Use hop count as a metric. • Use Bellman-Ford Algorithm. • Example: RIP, EIGRP. • Limitation: Can cause high traffic in large networks.	• Routers build a full map of the network. • Send updates only when changes occur. • Example: OSPF.

IGP (Interior Gateway Protocol)

- ⇒ Used for **routing within an Autonomous System (AS)**.
- ⇒ **AS**: Group of networks under one administration.

❖ Examples:

- **RIP v1**: Routing Information Protocol (Version 1)
- **IGRP**: Interior Gateway Routing Protocol
- **EIGRP**: Enhanced Interior Gateway Routing Protocol
- **OSPF**: Open Shortest Path First
- **IS-IS**: Intermediate System to Intermediate System

IGP (Interior Gateway Protocol)

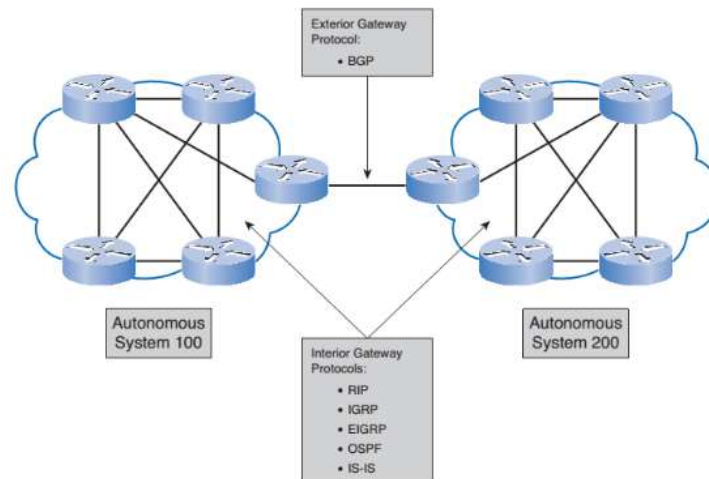
- ⇒ Used for **Routing between autonomous systems(AS)**
- ⇒ **AS:** Group of networks under one administration.

❖ Examples:

- **BGP:** Border Gateway Protocol

Autonomous System (AS)

- ⇒ A group of networks under **one administrative control**.
- ⇒ Each AS uses **its own internal routing policy**.



Administrative Distance (AD)

- **Trustworthiness** is used to describe **administrative distance (AD)**.
- **AD** defines the **preference of a routing source**.
- **AD values range from 0 to 255**.
- **Lower AD = more preferred** route.
- **AD of 0** is the **most preferred** and belongs only to **directly connected networks**.
- **AD of 0 cannot be changed**.

Default AD Values of Routing Protocols

Route Source	AD (Administrative Distance)
Connected	0
Static	1
EIGRP summary route	5
External BGP	20
Internal EIGRP	90
IGRP	100
OSPF	110
IS-IS	115
RIP	120
External EIGRP	170
Internal BGP	200

Routing Information Protocol (RIP)

- ⇒ Type: **Distance Vector Protocol**
- ⇒ Metric: Hop Count
- ⇒ Max Hop Count: 15 (16 = unreachable)
- ⇒ Updates: Every 30 seconds
- ⇒ Invalid route timeout: 180 seconds (if no update received)
- ⇒ Versions: RIPv1 and RIPv2

RIPv1 vs RIPv2

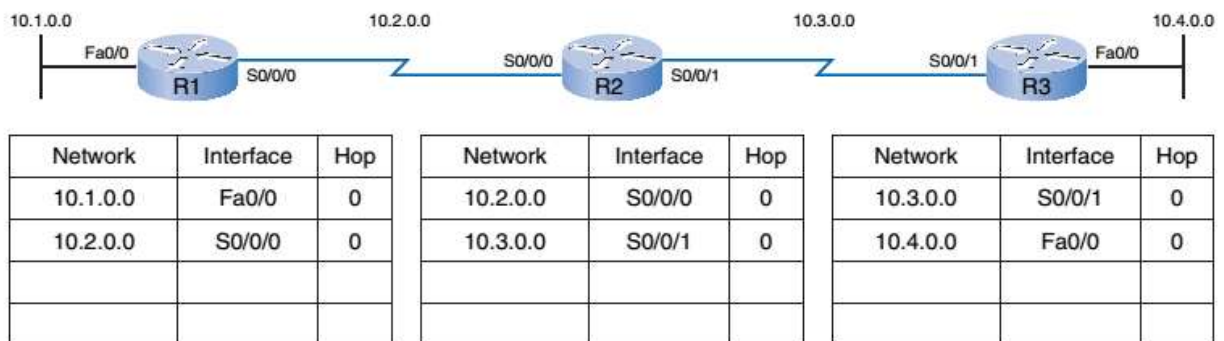
Feature	RIPv1	RIPv2
Addressing	Classful	Classless (VLSM supported)
Communication	Broadcast (255.255.255.255)	Multicast (224.0.0.9)
Updates	Sends v1, receives both v1 & v2 (ignores v2)	Sends & receives v2 only
Authentication	No	Yes

Routing table update

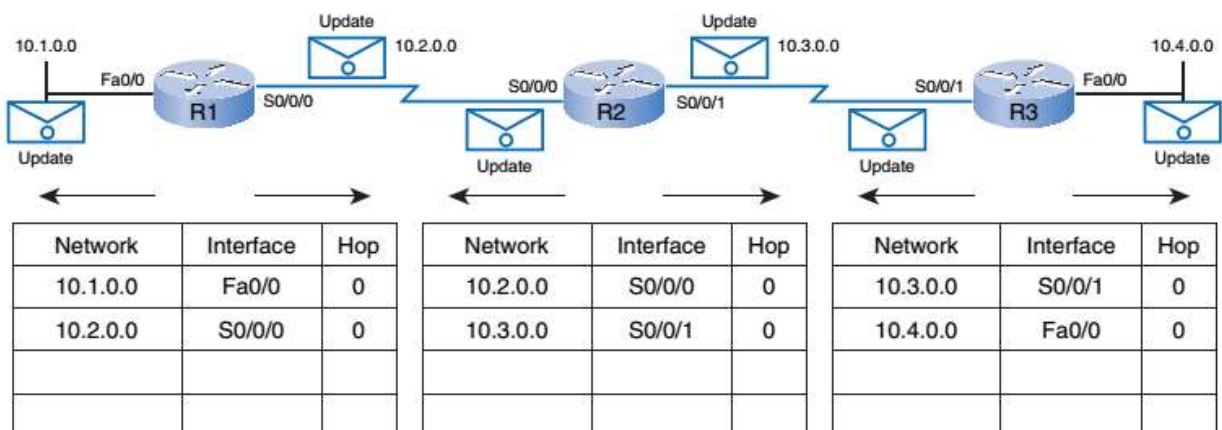
- **Cold Start:** Router starts and requests routing info.
- **Initial Exchange:** Routers share their tables.
- **Table Updating:** Updates happen every 30 seconds.
- **Next Update:** Routers re-evaluate and refresh routes.
- **New Route:** Added when a new network is discovered.
- **Removing Route:** Route deleted if not updated within timeout period.

⇒ An Example:

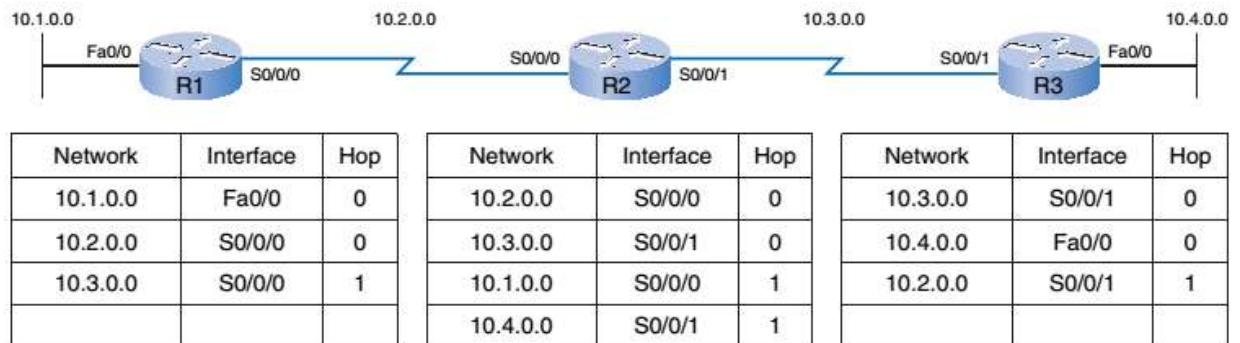
❖ **Cold Start:** Each router knows only directly connected networks



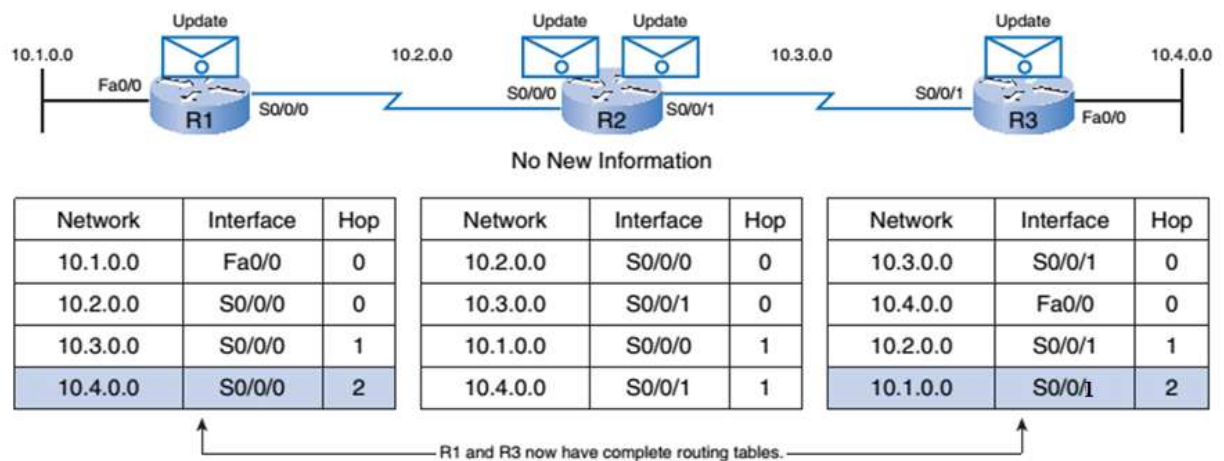
❖ **Initial Exchange:** Routers share their tables.



- ❖ **Table Updating:** Updates happen every 30 seconds.

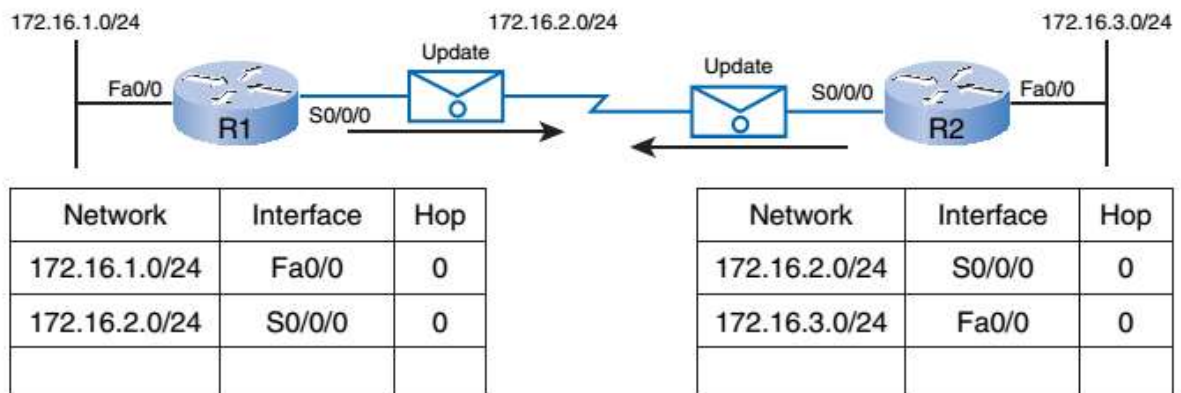


- ❖ **Next Update:** Routers re-evaluate and refresh routes.

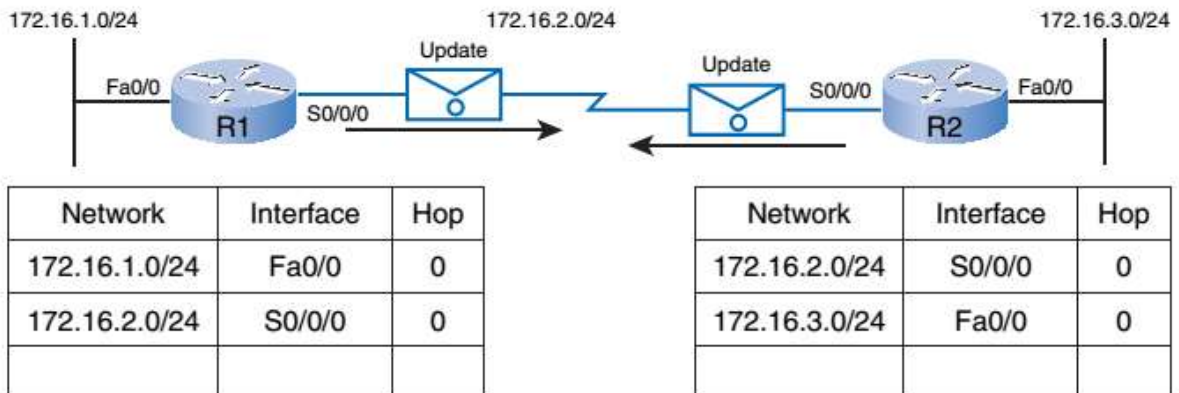


⇒ Another Example:

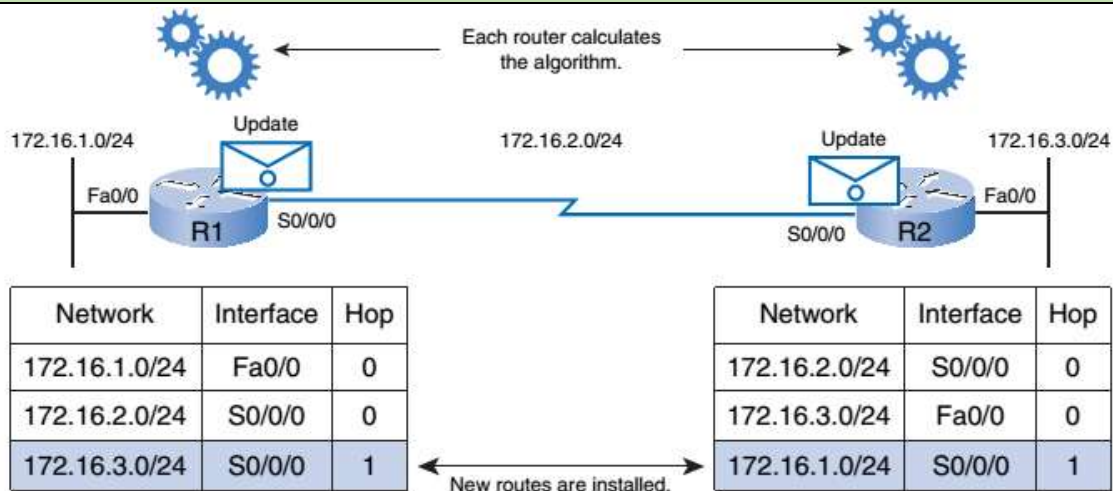
- ❖ **Cold Start:** Each router knows only directly connected networks



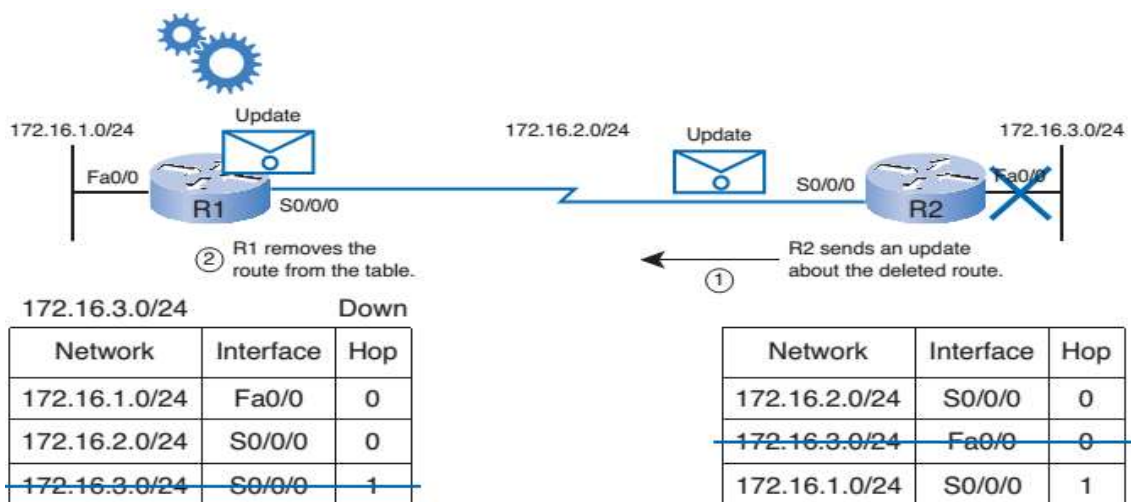
- ❖ **Initial Exchange:** Routers share their tables.



- ❖ **New Route:** Added when a new network is discovered.



- ❖ **Removing Route:** Route deleted if not updated within timeout period.



DHCP & ARP

What is DHCP?

- ⇒ **DHCP (Dynamic Host Configuration Protocol)** is a network protocol that automatically gives an IP address to each device (client) in a network.
- It helps devices connect to the network without manually configuring IP settings.
- ❖ **Example:** When you connect your laptop to Wi-Fi, the **DHCP server** gives your device an IP address automatically.

BOOTP (Bootstrap Protocol)

- An older protocol before DHCP.
- Assigns an IP address to devices when they boot up.
- Uses a **static table** — each MAC address is permanently linked to one IP.
- Cannot handle moving devices or temporary IPs.
- **DHCP improved BOOTP** by adding dynamic address allocation.

DHCP Address Allocation

- Static Allocation:**
 - IP is permanently assigned to a device (like BOOTP).
- Dynamic Allocation:**
 - IP is temporarily assigned from a **pool** for a fixed period (called **lease time**).

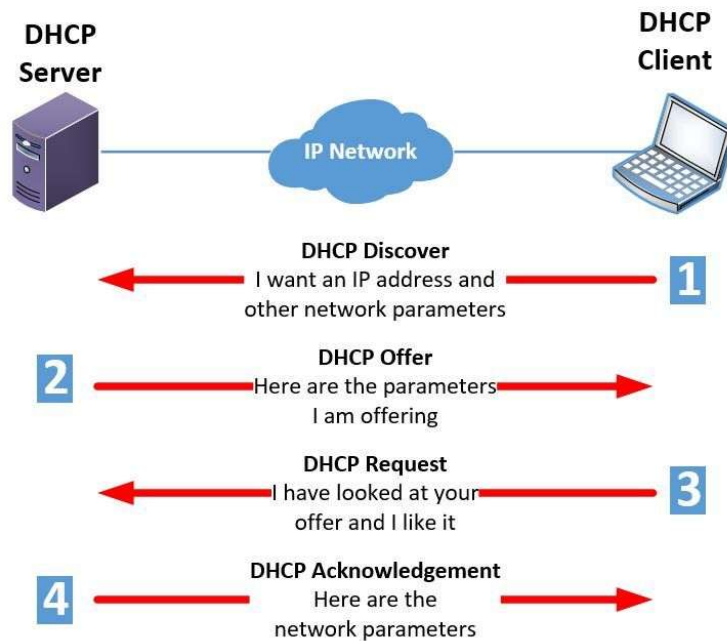
Steps of DHCP Operation

State	Description	Messages Used
INIT	Client starts and broadcasts a request for IP.	DHCPDISCOVER
SELECTING	Servers offer available IPs.	DHCPOFFER
REQUESTING	Client requests one IP offer.	DHCPREQUEST
BOUND	Client uses the IP until the lease expires.	DHCPACK
RENEWING	At 50% lease time, client requests renewal.	DHCPREQUEST + DHCPACK
REBINDING	At 87.5% lease time, client tries again. If no response, it restarts.	DHCPNACK or DHCPACK

❖ **Example:** If lease time (LT) = 1 hour 28 minutes

- Renewing timer = 50% → **44 minutes**
- Rebinding timer = 87.5% → **~1 hour 16 minutes**

Steps of DHCP



1 DHCP Discover—The client broadcasts a request to find available DHCP servers, asking for an IP address and other configuration details.

2 DHCP Offer – The DHCP server responds with an offer that includes an available IP address, subnet mask, gateway, and DNS settings.

3 DHCP Request – The client accepts the offer by sending a request message back to the server.

4 DHCP Acknowledgement – The server confirms and finalizes the assignment, and the client is officially connected to the network.

✂ **Problem1:** A Client pc is looking for an **ip address**. The DHCP server has offered three ip addresses (**207.0.0.6/24, LT- 1 hr 14 min**), (**207.0.0.8/24, LT- 1 hr 28 min**), (**207.0.0.11/24, LT- 1 hr 32 min**). The Client has chosen the **2nd ip address**. If the client asks for extended LT of 22 minutes, What is the new timer of **RENEWING and REBINDING** states? While calculating timer for REBINDING state, consider the lease time was not extended in RENEWING state. [LT means lease time]

Is given,

- ⇒ Chosen IP address: 207.0.0.8/24
- ⇒ Original Lease Time (LT): 1 hr 28 min = 88 minutes
- ⇒ Extended Lease Time: +22 minutes
- ⇒ New Total Lease Time: 88+22 = 110 min

We Know,

RENEWING (T1) time:

$$\begin{aligned} T1 &= 50\% \times \text{Total lease time} \\ &= 50\% \times 110 \\ &= 55 \text{ min} \end{aligned}$$

So, the client will start **RENEWING** after **55 minutes**.

REBINDING (T2) time:

The problem states: “while calculating timer for REBINDING state, consider the lease time was **not extended** in RENEWING state.”

That means for **T2**, we use the **original lease time** (before extension):

$$\begin{aligned} T2 &= 87.5\% \times \text{lease time} \\ &= 87.5\% \times 88 \\ &= 77 \text{ min} \end{aligned}$$

So, **REBINDING** starts at **77 minutes** (based on the original LT).

What is ARP (Address Resolution Protocol)?

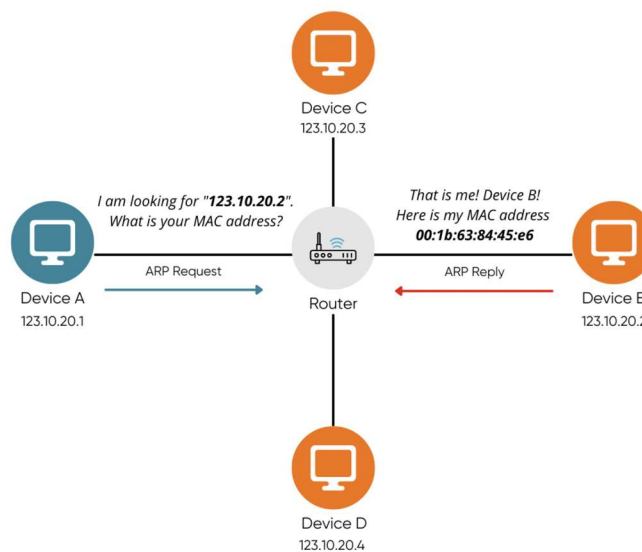
- ⇒ **ARP** helps a device find the **MAC address** (physical address) of another device using its **IP address** in the same network.

❖ Example Scenario:

Device	IP Address	MAC Address
 PC-A	123.10.20.1	00:1A:2B:3C:4D:5E
 PC-B	123.10.20.2	00:1B:63:84:45:E6
 PC-C	123.10.20.3	AC:DE:48:00:11:22
 PC-C	123.10.20.4	D4:6D:6D:AA:BB:CC

If PC-A wants to send data to PC-B but only knows its IP → ARP helps find PC-B's MAC address.

How ARP Works

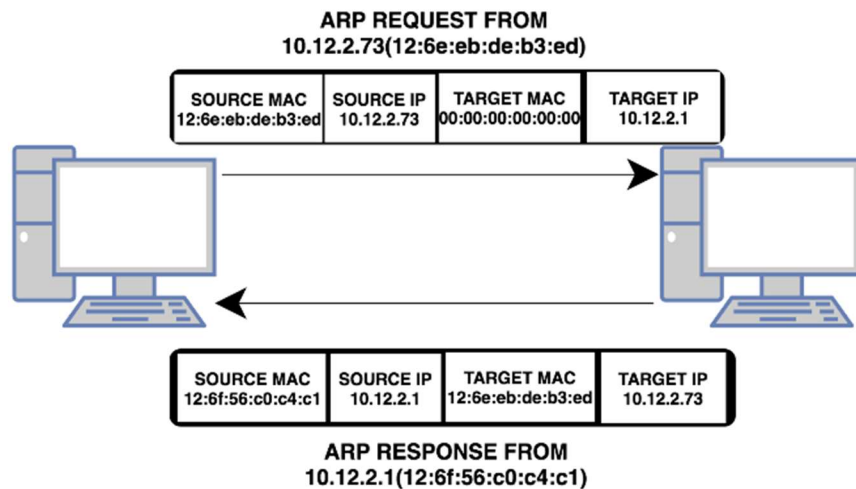


Step 1: ARP Request

- PC-A broadcasts a message asking: “Who has IP **123.10.20.2**? Tell me your MAC address.”
- Destination MAC: **00:1A:2B:3C:4D:5E** (broadcast).

Step 2: ARP Reply

- Only PC-B responds: “I am 123.10.20.2. My MAC is **00:1B:63:84:45:E6**.”
- PC-A saves this info in its **ARP cache**.



ARP Cache Timeout

- ARP cache stores IP–MAC pairs temporarily.
- Helps avoid sending ARP requests repeatedly.
- Entries **expire after a few minutes** (typically **2–20 mins**) to keep info updated.

Advantages & Disadvantages

Advantages	Disadvantages
Reduces manual IP setup (DHCP).	ARP can be used for attacks (ARP spoofing).
Ensures IP reuse & efficiency.	DHCP failure can stop network access.
Simplifies communication setup.	ARP adds some broadcast traffic.

IP ADDRESSING – 1

What is Addressing?

- ⇒ To send data from one device to another, each device must have an **address** to identify it.

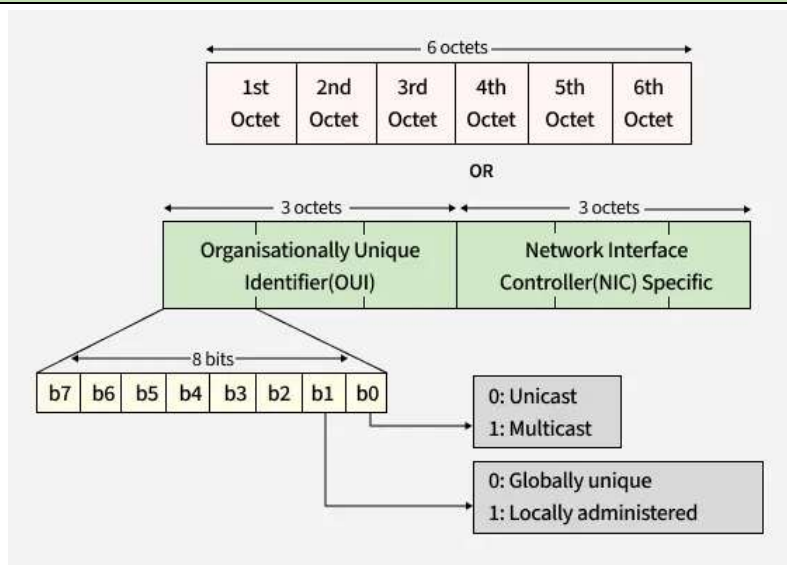
❖ Types of Addresses:

Type	Description	Example / Size
Physical Address (MAC)	The address of a node as defined by its LAN. Used for local communication (within same network).	6 bytes (48 bits) e.g., CA-18-85-0C-B7-2C
Logical Address (IP)	Used for communication between different networks (Internet). Each host has a unique 32-bit IP address.	32 bits e.g., 192.168.10.25
Port Address	Identifies a process or service (like Email, FTP). Assigned by IANA .	16 bits e.g., Port 25 (SMTP)

Physical Address (MAC)

- ⇒ Built into the **Network Interface Card (NIC)**.
- ⇒ **Permanent** (can't be changed easily).
- ⇒ Divided into:
 - **Organizationally Unique Identifier (OUI)** – first 3 bytes.
 - **Device ID** – last 3 bytes.
- ⇒ Used only **within LAN**.

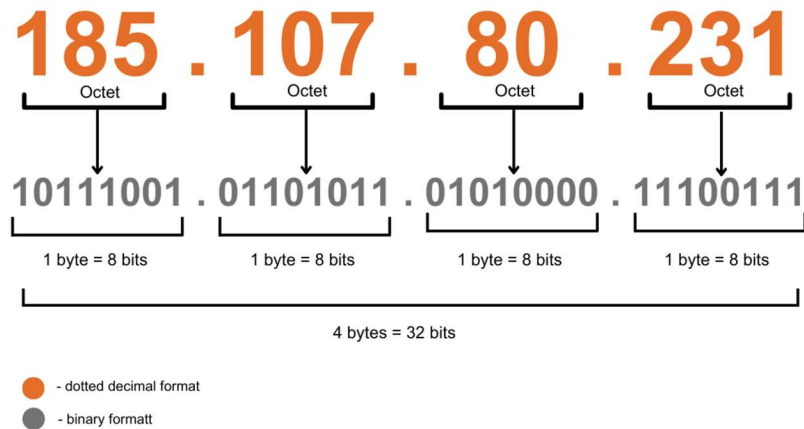
Media Access Control (MAC):



Logical Address (IP)

- ⇒ Needed for **internetwork** communication.
- ⇒ **Universal** addressing system.
- ⇒ **Can be changed** depending on the network.
- ⇒ No two public IPs are the same.

IPv4 Address Format



Port Address

- ⇒ Used for **specific processes**.
- ⇒ Example:
 - HTTP → Port 80
 - FTP → Port 21
 - Email → Port 25

Port Number	Process Name	Protocol Used	Description
20	FTP-DATA	TCP	File transfer---data
21	FTP	TCP	File transfer---control
22	SSH	TCP	Secure Shell
23	TELNET	TCP	Telnet
25	SMTP	TCP	Simple Mail Transfer Protocol
53	DNS	TCP & UDP	Domain Name System
69	TFTP	UDP	Trivial File Transfer Protocol
80	HTTP	TCP & UDP	Hypertext Transfer Protocol
110	POP3	TCP	Post Office Protocol 3
123	NTP	TCP	Network Time Protocol
143	IMAP	TCP	Internet Message Access Protocol
443	HTTPS	TCP	Secure implementation of HTTP

Binary Decimal conversion

➔ An **8-bit binary number** consists of 8 digits (bits), each representing a power of 2.

Digit	x8	x7	x6	x5	x4	x3	x2	x1
Position of digit (i)	8	7	6	5	4	3	2	1
Contribution of the digit (2^{i-1})	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Decimal contribution	128	64	32	16	8	4	2	1

 **Problem1:** Convert the following **8-bit binary numbers** into their **decimal equivalents**:

Sl. No.	Binary Number (8-bit)	Decimal Equivalent
1	1100 1010	_____
2	1110 0001	_____
3	1010 1111	_____

For 1100 1010:

[illegible]

For 1110 0001:

[illegible]

100% MONEY BACK GUARANTEE
 30 DAY MONEY BACK GUARANTEE

[illegible]

 Problem2: Convert the following **decimal equivalents** into their **8-bit binary numbers**:

Sl. No.	Binary Number (8-bit)	Decimal Equivalent
1	_____	210
2	_____	50
3	_____	65

For 210:

[illegible]

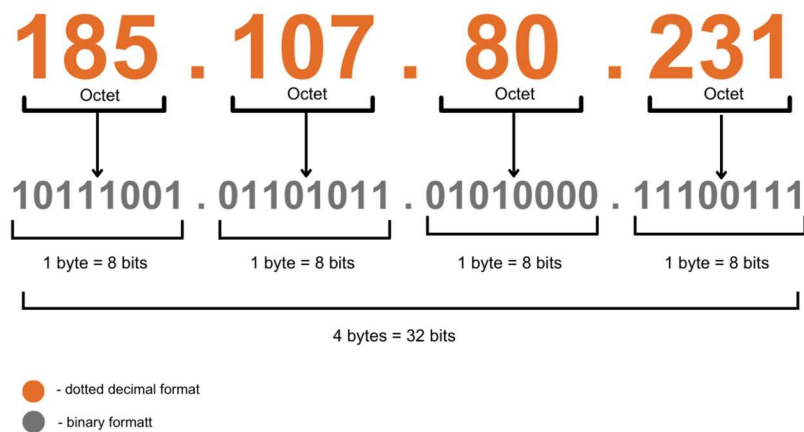
101 30.
XXXXXXXXXX

[illegible]

IP Address Basics

- **Full form:** Internet Protocol Address
- **Length:** 32 bits
- **Divided into:** 4 octets (8 bits each)
- **Written as:** dotted decimal notation
 - Example:
Binary → 10000000.00001011.00000011.00011111
Decimal → 128.11.3.31

IPv4 Address Format



Address Space

→ Total number of possible addresses = 2^n (where n = number of bits)

For 32-bit IPV4: $2^{32} = 4,294,967,296$ total IPs.

🔗 **Problem3:** Change the following IPv4 addresses from **binary notation** to **dotted-decimal notation**.

a. 10000001 00001011 00001011 11101111

b. 11000001 10000011 00011011 11111111

c. 10001000 10010011 11100000 00000110

For 10000001 00001011 00001011 11101111:

Octet	1	2	3	4
Digit	10000001	00001011	00001011	11101111
Decimal contribution	$128+1=129$	$8+2+1=11$	$8+2+1=11$	$128+64+32+8+4+2+1=239$
Dotted-decimal notation	129.11.11.239			

For 11000001 10000011 00011011 11111111:

Octet	1	2	3	4
Digit	11000001	10000011	00011011	11111111
Decimal contribution	$128+64+1=193$	$128+2+1=131$	$16+8+2+1=27$	$128+64+32+16+8+4+2+1=255$
Dotted-decimal notation	193.131.27.255			

For 10001000 10010011 11100000 00000110:

Octet	1	2	3	4
Digit	10001000	10010011	11100000	00000110
Decimal contribution	$128+8=136$	$128+16+2+1=147$	$128+64+32=224$	$4+2=6$
Dotted-decimal notation	136.147.224.6			



a. 111.56.45.78

b. 221.34.7.82

For 111.56.45.78:

Octet	1	2	3	4
Decimal contribution	111	56	45	78

For Octet 1: 111:

[illegible]

For Octet 1: 56:

[illegible]

For Octet 1: 45:

Decimal contribution	128	64	32	16	8	4	2	1
Position of digit (i)	8	7	6	5	4	3	2	1
Compare value	$45 < 128$	$45 < 64$	$45 \geq 32$	$13 < 16$	$13 \geq 8$	$5 \geq 4$	$1 < 2$	$1 \geq 1$
New value	-	-	$45-32=13$	-	$13-8=5$	$5-4=1$	-	$1-1=0$
Digit	0	0	1	0	1	1	0	1
Binary of 45	111 = (00101101)₂							

For Octet 1: 78:

Decimal contribution	128	64	32	16	8	4	2	1
Position of digit (i)	8	7	6	5	4	3	2	1
Compare value	$78 < 128$	$78 \geq 64$	$14 < 32$	$14 < 16$	$14 \geq 8$	$6 \geq 4$	$0 \geq 2$	$0 < 1$
New value	-	$78-64=14$	-	-	$14-8=6$	$6-4=2$	$2-2=0$	-
Digit	0	1	0	0	1	1	1	0
Binary of 78	111 = (01001110)₂							

So, Binary notation of 111.56.45.78:

Octet	1	2	3	4
Decimal contribution	111	56	45	78
Digit	01101111	000111000	00101101	01001110
Binary notation	01101111. 000111000. 00101101. 01001110			

✂ **Problem5:** Find the error if any in the following IPV4 address

a. 11.56.045.68

b. 221.34.7.8.39

(a)

Step 1: Count the octets

- There are **4 octets** → Correct count

Step 2: Check each octet range (0–255)

- 11 → valid
- 56 → valid
- 045 → invalid **notation** — IPv4 octets **must not have leading zeros** (they can be misinterpreted as octal numbers).
- 68 → valid

Error: Octet **045** has a **leading zero** → invalid IPv4 format.

Correct form: 11.56.45.68

(b)

Step 1: Count the octets

- There are **5 octets** → Invalid — an IPv4 address must have **exactly 4 octets**.

Error: Contains **too many octets (5)**.

Should have only 4 numbers separated by 3 dots.

✂ **Problem6:** Find the number of addresses in a range if the first address is **146.102.29.0** and the last address is **146.102.32.255**.

Solution:

$$\text{Number of Addresses} = (\text{Last Address} - \text{First Address}) + 1$$

$$\text{Address Value} = A \times 256^3 + B \times 256^2 + C \times 256^1 + D \times 256^0$$

$$\begin{aligned} \therefore \text{Number of addresses} &= (0 \times 256^3 + 0 \times 256^2 + 3 \times 256^1 + 255 \times 256^0) + 1 \\ &= 1024 \end{aligned}$$

✂ **Problem7:** The first address in a range of addresses is **14.11.45.96**. If the number of addresses in the range is **32**, what is the last address?

Solution:

$$\text{Last Address} = \text{First Address} + (\text{Number of Addresses} - 1)$$

$$\text{Last address} = (14.11.45.96 + 0.0.0.31)_{256} = \mathbf{14.11.45.127}$$

✂ **Problem8:** The first address in a range of addresses is **192.168.10.128**. If the number of addresses in the range is **64**, what is the last address?

Solution:

$$\text{Last Address} = \text{First Address} + (\text{Number of Addresses} - 1)$$

$$\text{Last address} = (192.168.10.128 + 0.0.0.63)_{256} = \mathbf{192.168.10.193}$$

✂ **Problem9:** Find the number of addresses in a range if the first address is **145.102.29.0** and the last address is **145.104.33.255**.

Solution:

$$\text{Number of Addresses} = (\text{Last Address} - \text{First Address}) + 1$$

$$\text{Address Value} = A \times 256^3 + B \times 256^2 + C \times 256^1 + D \times 256^0$$

$$\begin{aligned} \therefore \text{First Address Value} &= 145 \times 256^3 + 102 \times 256^2 + 29 \times 256^1 + 0 \times 256^0 \\ &= 2439388416 \end{aligned}$$

$$\begin{aligned} \therefore \text{Last Address Value} &= 145 \times 256^3 + 104 \times 256^2 + 33 \times 256^1 + 255 \times 256^0 \\ &= 2439520767 \end{aligned}$$

$$\begin{aligned} \therefore \text{Number of Addresses} &= (2439520767 - 2439388416) + 1 \\ &= 132352 \end{aligned}$$

IP ADDRESSING - 11

Classful Addressing

	First byte	Second byte	Third byte	Fourth byte
Class A	0			
Class B	10			
Class C	110			
Class D	1110			
Class E	1111			

a. Binary notation

	First byte	Second byte	Third byte	Fourth byte
Class A	0-127			
Class B	128-191			
Class C	192-223			
Class D	224-239			
Class E	240-255			

b. Dotted-decimal notation

✂ **Problem1:** Find the Class of the following dotted decimal IPv4 address.

1. 192.168.1.10
2. 10.10.200.6
3. 172.15.165.1
4. 230.10.65.30

Solution:

IPv4 address	Range	Class
192.168.1.10	192-223	Class C
10.10.200.6	0-127	Class A
172.15.165.1	128-191	Class B
230.10.65.30	224-239	Class D

✂ **Problem2:** Find the Class of the following dotted decimal IPv4 address.

- a. 11110001 00001011 00001011 11101111
- b. 11000001 10000011 00011011 11111111
- c. 10001000 10010011 11100000 00000110
- d. 11101000 10010011 11100000 00000110

Solution:

IPv4 address	Class
11110001 00001011 00001011 11101111	Class E
11000001 10000011 00011011 11111111	Class C
10001000 10010011 11100000 00000110	Class B
11101000 10010011 11100000 00000110	Class D

IPv4 Address Classes

Address Class	1st Octet Range (Decimal)	1st Octet Bits (Blue Dots Do Not Change)	Network (N) and Host (H) Portion	Default Mask (Decimal)	Number of Possible Networks and Hosts per Network
A	0–127	00000000 – 01111111	N.H.H.H	255.0.0.0	128 Nets (2^7) 16,777,214 hosts ($2^{24} - 2$)
B	128–191	10000000 – 10111111	N.N.H.H	255.255.0.0	16,384 Nets (2^{14}) 65,534 hosts ($2^{16} - 2$)
C	192–223	11000000 – 11011111	N.N.N.H	255.255.255.0	2,097,152 Nets (2^{21}) 254 hosts ($2^8 - 2$)
D	224–239	11100000 – 11101111	NA (Multicast)	–	–
E	240–255	11110000 – 11111111	NA (Experimental)	–	–

Subnet Mask (Slash Notation)

→ A 32-bit number showing which bits belong to the **network** and which to the **host**.

→ All **network bits** = 1, all **host bits** = 0

Address Class	Network (N) Host (H) Portion	Subnet Mask (Decimal)	Subnet Mask (in Binary)	Slash Notation or Prefix
A	N.H.H.H	255.0.0.0	11111111.00000000.00000000.00000000	/8
B	N.N.H.H	255.255.0.0	11111111.11111111.00000000.00000000	/16
C	N.N.N.H	255.255.255.0	11111111.11111111.11111111.00000000	/24

❖ Example1:

IP Address	Subnet Mask Using	IP Address	Subnet Mask Using
10.10.10.1	255.0.0.0 Same N/W	10.10.10.1	255.255.255.0 Different N/W
10.10.20.1		10.10.20.1	
172.16.200.1	255.255.0.0 Same N/W	172.16.200.1	255.255.255.0 Different N/W
172.16.165.2		172.16.165.2	
192.168.10.1	255.255.255.0 Same N/W	192.168.10.1	255.0.0.0 Same N/W
192.168.10.2		192.168.10.2	

Special Purpose IPs

IP Address	Use
0.0.0.0	Used for DHCP discovery or default route
127.0.0.1	Loopback (self-test)
255.255.255.255	Limited broadcast (same network)
x.x.x.0	Network address (Class C example)
x.x.x.255	Broadcast address (Class C example)

Different Ways to Transmit IPv4

⇒ IPv4 packets can be carried from one device to another using **different transmission methods** depending on the type of network and addressing.

1. Unicast

- ◆ **One-to-One** communication
- ◆ IPv4 packet is sent from **one sender** → **one specific receiver**
- ◆ Most common type (web browsing, file transfer)

Example: Your PC → Google Server

2. Broadcast

- ◆ **One-to-All** communication
- ◆ Sender sends IPv4 packet to **all devices in the network**
- ◆ Uses special address: **255.255.255.255** or network broadcast address

Example: ARP Request → sent to every device in the LAN

3. Multicast

- ◆ **One-to-Many (selected group)**
- ◆ Packet is delivered to **specific group members**
- ◆ Uses IPv4 multicast range: **224.0.0.0 – 239.255.255.255**

Example: Live streaming → only subscribed devices receive it

Broadcast Types

Type	Description	Example
Limited Broadcast	Sent to all hosts in the same network	255.255.255.255
Direct Broadcast	Sent to all hosts in another network	For 20.0.0.0 → 20.255.255.255

Network, Host, and Broadcast Ips

⇒ A network has:

1. **Network IP Address** → first IP (all host bits = 0)
2. **Broadcast IP Address** → last IP (all host bits = 1)
3. **Host IPs** → all in between (usable addresses)

$$\text{Usable Host IPs} = \text{Total Host IPs} - 2 \quad [1 \text{ reserved for network, } 1 \text{ for broadcast}]$$

❖ **Example:** For 192.100.12.110 →

- Network IP = **192.100.12.0**
- Broadcast IP = **192.100.12.255**
- Usable hosts = $2^8 - 2 = 254$

Address Block Info

Class	n	k	No. of Networks (Blocks) (2^{n-k})	No. of Hosts in each Network (2^{32-n})
Class A	8	1	128	16,777,216
Class B	16	2	16,384	65,536
Class C	24	3	2,097,152	256

Wastage in Classful System

- ⇒ Fixed class sizes caused **IP wastage**.
- ⇒ Example: A small company with **50 PCs needed Class C (256 IPs)** but got only one block. → Remaining unused addresses wasted.
- ⇒ **Solution: Classless Addressing (CIDR) and Subnetting** (covered later).

✂ **Problem3:** Find the **network and broadcast** IP addresses of the network which uses the followings as host address. Also **Find the range of hosts of each network** as well.

1. 172.10.12.10
2. 204.130.120.10
3. 100.13.10.1
4. 10.5.3.5
5. 192.168.10.15

Solution:

Identify the classes:

IPv4 address	Range	Class
172.10.12.10	128-191	Class B
204.130.120.10	192-223	Class C
100.13.10.1	0-127	Class A
10.5.3.5	0-127	Class A
192.168.10.15	192-223	Class C

For 172.10.12.10	For 204.130.120.10
Class: B → Mask: 255.255.0.0 (/16) Network: 172.10.0.0 Broadcast: 172.10.255.255 Usable host range: 172.10.0.1 — 172.10.255.254 Usable hosts: $2^{16} - 2 = 65,534$	Class: C → Mask: 255.255.255.0 (/24) Network: 204.130.120.0 Broadcast: 204.130.120.255 Usable host range: 204.130.120.1 — 204.130.120.254 Usable hosts: $2^8 - 2 = 254$
For 100.13.10.1	For 10.5.3.5
Class: A → Mask: 255.0.0.0 (/8) Network: 100.0.0.0 Broadcast: 100.255.255.255 Usable host range: 100.0.0.1 — 100.255.255.254 Usable hosts: $2^{24} - 2 = 16,777,214$	Class: A → Mask: 255.0.0.0 (/8) Network: 100.0.0.0 Broadcast: 100.255.255.255 Usable host range: 100.0.0.1 — 100.255.255.254 Usable hosts: $2^{24} - 2 = 16,777,214$

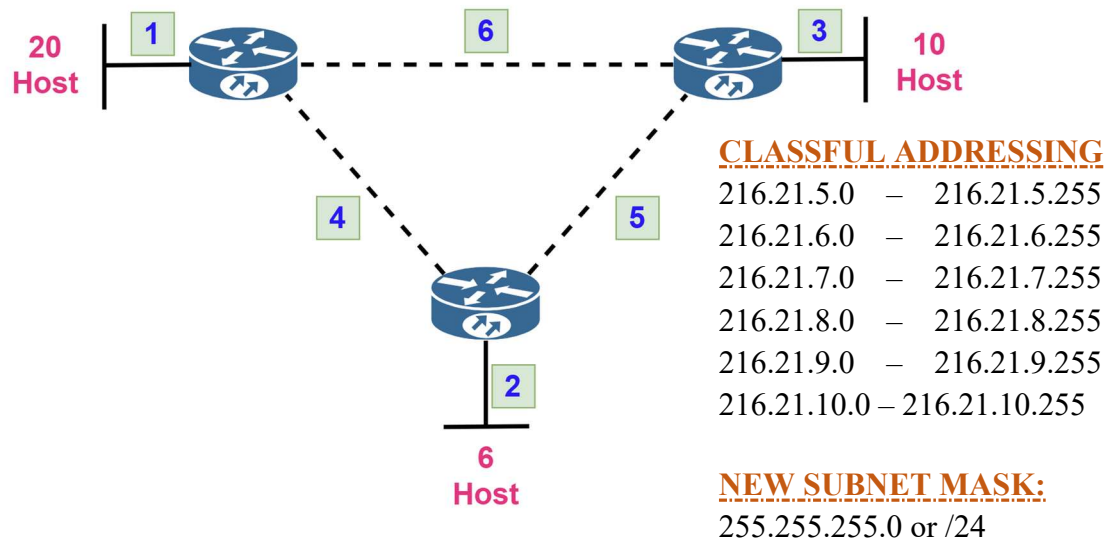
VARIABLE LENGTH SUBNET MASK

FLSM – Fixed Length Subnet Mask

⇒ FLSM means **all subnets have the SAME size**.

- Same **subnet mask**
- Same **number of hosts** in each subnet
- Simple and easier than VLSM

Also called: **Classful subnetting** or **Equal subnetting**



What is the problem with a large network?

- ⇒ Hard to manage
- ⇒ Less security (everyone can reach everyone)
- ⇒ Large broadcast domain → more traffic
- ⇒ Many IP addresses get wasted

What is Subnetting?

- ⇒ Breaking a **large network** into **small sub-networks**.
- ⇒ Helps reduce broadcast domain → faster network → more security.

But, **Classful subnetting** gives **equal IPs** to all subnets → still wasteful.

LESTN (Variable Length Subnet Mask)

- ⇒ No waste of IPs.

Simple idea: *Bigger subnet $\Rightarrow$ larger IP block*

VENI Steps

Step 1: List all networks in **descending order** based on required hosts (largest to smallest).

Step 2: Find how many **host bits (h)** each subnet needs using:

$$2^H \geq \text{hosts needed}$$

Step 3: Calculate the **subnet mask** for each subnet:

Prefix = 32 – H

Step 4: Starting from the beginning of the given IP block, **assign subnets sequentially**.

For each subnet, determine:

- Network address
- First host
- Last host
- Broadcast address

🔗 **Problem1:** Suppose that we have three networks: A, B and C with IP requirements 20, 20 and 60. If You are given an IP block 192.168.10.0, allocate IPs performing subnetting.

Subnet	No. of IPs Required	How Many Bits to Borrow	No. of Allocated IPs	No. of Host Bits (H)	No. of Net. Bits (N=32-H)	Subnet Mask	Allocated IP Range
C	60	$2^6 \geq 60$	64	H = 6	N = 32 – 6 = 26	255.255.255.192	192.168.10.0 – 192.168.10.63 / 26
A	20	$2^5 \geq 20$	32	H = 5	N = 32 – 5 = 27	255.255.255.224	192.168.10.64 – 192.168.10.95 / 27
B	20	$2^5 \geq 20$	32	H = 5	N = 32 – 5 = 27	255.255.255.224	192.168.10.96 – 192.168.10.127 / 27

For subnet C

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

For subnet A

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

For subnet B

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

✂ **Problem2:** Suppose that we have six networks: A, B, C, D, E, F, G with IP requirements 56, 4, 4, 4, 5, 21 and 26. If You are given an IP block 192.168.5.0, allocate IPs performing subnetting.

Digit	x8	x7	x6	x5	x4	x3	x2	x1
Position of digit (i)	8	7	6	5	4	3	2	1
Contribution of the digit (2^{i-1})	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Decimal contribution	128	64	32	16	8	4	2	1

Subnet	No. of IPs Required	How Many Bits to Borrow	No. of Allocated IPs	No. of Host Bits (H)	No. of Net. Bits (N=32-H)	Subnet Mask	Allocated IP Range
A	56	$2^6 \geq 56$	64	H = 6	N = 32 - 6 = 26	255.255.255.192	192.168.5.0 – 192.168.5.63 / 26
G	26	$2^5 \geq 26$	32	H = 5	N = 32 - 5 = 27	255.255.255.224	192.168.5.64 – 192.168.5.95 / 27
F	21	$2^5 \geq 21$	32	H = 5	N = 32 - 5 = 27	255.255.255.224	192.168.5.96 – 192.168.10.127 / 27
E	5	$2^3 \geq 5$	8	H = 3	N = 32 - 3 = 29	255.255.255.248	192.168.5.128 – 192.168.10.135 / 29
B	4	$2^2 \geq 4$	4	H = 2	N = 32 - 2 = 30	255.255.255.252	192.168.5.136 – 192.168.10.139 / 30
C	4	$2^2 \geq 4$	4	H = 2	N = 32 - 2 = 30	255.255.255.252	192.168.5.140 – 192.168.10.143 / 30
D	4	$2^2 \geq 4$	4	H = 2	N = 32 - 2 = 30	255.255.255.252	192.168.5.144 – 192.168.10.147 / 30

VLSM Efficiency

For VLSM:

- **Allocated IPs (VLSM):** $64 + 32 + 32 + 8 + 4 + 4 + 4 = 148$ total addresses allocated
- **Required (usable) hosts:** $56 + 26 + 21 + 5 + 4 + 4 + 4 = 120$
- **Unused addresses inside allocated subnets:** $148 - 120 = 28$ unused addresses

∴ **Percentage of unused addresses (within allocated space):**

$$= \frac{148 - 120}{148} \times 100\% \approx 19\%$$

For FLSM:

- **Allocated IPs (FLSM):** $255 \times 7 = 1785$ total addresses allocated
- **Required (usable) hosts:** $56 + 26 + 21 + 5 + 4 + 4 + 4 = 120$
- **Unused addresses inside allocated subnets:** $1785 - 120 = 1665$ unused addresses

∴ **Percentage of unused addresses (within allocated space):**

$$= \frac{1785 - 120}{1785} \times 100\% \approx 93\%$$

⇒ **Percentage of unused IP address (for the given network)**

Classful Addressing	VLSM
93%	19%

✂ **Problem5:** Suppose that we have six networks: A, B, C, D, E and F with **IP requirements** 120, 400, 9, 40, 32, and 7, respectively. If You are given an IP block 210.3.0.0, allocate IPs performing subnetting.

➔ **Solution:**

For subnet B:

1 0 0 0 0 0 0 0 0 0 0

For subnet A:

1 0 0 0 0 0 0 0 0

For subnet D:

1 0 0 0 0 0 0 0 0

For subnet E:

1 0 0 0 0 0 0 0 0

For subnet C:

1 0 0 0 0 0 0 0 0

For subnet F:

1 0 0 0 0 0 0 0 0

Subnet	No. of IPs Required	How Many Bits to Borrow	No. of Allocated IPs	No. of Host Bits (H)	No. of Net. Bits (N=32-H)	Subnet Mask	Allocated IP Range
B	400	$2^9 > 400$	512	H = 9	N = 32 - 9 = 23	255.255.254.0	210.3.0.0 – 210.3.1.255 / 23
A	120	$2^7 \geq 120$	128	H = 7	N = 32 - 7 = 25	255.255.255.128	210.3.2.0 – 210.3.2.127 / 25
D	40	$2^6 \geq 40$	64	H = 6	N = 32 - 6 = 26	255.255.255.192	210.3.2.128 – 210.3.2.191 / 26
E	32	$2^5 \geq 32$	32	H = 5	N = 32 - 5 = 27	255.255.255.224	210.3.2.192 – 210.3.2.223 / 27
C	9	$2^4 \geq 9$	16	H = 4	N = 32 - 4 = 28	255.255.255.240	210.3.2.224 – 210.3.2.239 / 28
F	7	$2^3 \geq 7$	8	H = 3	N = 32 - 3 = 29	255.255.255.248	210.3.2.240 – 210.3.2.247 / 29

🔗 **Problem6:** Convert the following prefix in Subnet mask-

1. /16
2. /23
3. /25

➔ **Solution:**

For subnet /16: Binary format

1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

SO,

Octet	1	2	3	4
Digit	11111111	11111111	00000000	00000000
Decimal contribution	255	255	0	0
Dotted-decimal notation	255.255.0.0			

For subnet /23: Binary format

1 0 0 0 0 0 0 0 0 0 0

SO,

Octet	1	2	3	4
Digit	11111111	11111111	11111110	00000000
Decimal contribution	255	255	254	0
Dotted-decimal notation	255.255.254.0			

For subnet /25: Binary format

1 0 0 0 0 0 0 0 0 0 0

SO,

Octet	1	2	3	4
Digit	11111111	11111111	11111111	10000000
Decimal contribution	255	255	255	128
Dotted-decimal notation	255.255.255.128			

✂ **Problem7:** Convert the following Subnet mask in prefix-

1. 255.255.255.0
2. 255.255.128.0
3. 255.255.255.192

➔**Solution:**

For subnet 255.255.255.0:

Octet	1	2	3	4
Decimal contribution	255	255	255	0
Digit	11111111	11111111	11111111	00000000
Binary notation	11111111. 11111111. 11111111.00000000			
∴ Prefix	/24			

For subnet 255.255.128.0:

Octet	1	2	3	4
Decimal contribution	255	255	128	0
Digit	11111111	11111111	10000000	00000000
Binary notation	11111111. 11111111. 10000000. 00000000			
∴ Prefix	/17			

For subnet 255.255.255.192:

Octet	1	2	3	4
Decimal contribution	255	255	255	0
Digit	11111111	11111111	11111111	11000000
Binary notation	11111111. 11111111. 11111111. 11000000			
∴ Prefix	/26			

✂ **Problem8:** There are four networks: A, B, C and D with **60, 31, 530, 35 hosts**, respectively. If You are given an **IP block 10.10.0.0**, find the **subnet mask, network IP, broadcast IP and valid host IP range** for each network in such a way that the IP wastage is minimum

→ **Solution:**

For subnet C:

[illegible]

For subnet A:

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

For subnet D:

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

For subnet B:

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Subnet	No. of IPs Required	How Many Bits to Borrow	No. of Allocated IPs	No. of Host Bits (H)	No. of Net. Bits (N=32-H)	Subnet Mask	Allocated IP Range
C	530	$2^{10} - 2 \geq 530$	1024	H = 10	N = 32 - 10 = 22	255.255.252.0	10.10.0.0 – 10.10.3.255 / 22
A	60	$2^6 - 2 \geq 64$	64	H = 6	N = 32 - 6 = 26	255.255.255.192	10.10.4.0 – 10.10.4.63/ 26
D	35	$2^6 - 2 \geq 35$	64	H = 6	N = 32 - 6 = 26	255.255.255.192	10.10.4.64 – 10.10.4.127 / 26
B	31	$2^6 - 2 \geq 31$	64	H = 6	N = 32 - 6 = 26	255.255.255.192	10.10.4.128 – 10.10.4.191 / 26

Subnet Mask	Network IP	Broadcast IP	Valid Host IP Range
255.255.252.0	10.10.0.0	10.10.3.255	10.10.0.1 – 10.10.3.254 / 22
255.255.255.192	10.10.4.0	10.10.4.63	10.10.4.1 – 10.10.4.62/ 26
255.255.255.192	10.10.4.64	10.10.4.127	10.10.4.65 – 10.10.4.126 / 26
255.255.255.192	10.10.4.128	10.10.4.191	10.10.4.129 – 10.10.4.190 / 26

🔗 **Problem9:** Find the **sub network address**, **broadcast IP**, **subnet mask**, and **valid IP range** for the flowing IP address,

1. 101.88.220.31/17
2. 200.20.20.189/23

→ Solution:

For 101.88.220.31/17:

[illegible]

SO₂

Subnet Mask	Network Range	Network IP	Broadcast IP	Valid Host IP Range
255.255.128.0	101.88.0.0 – 101.88.127.255 101.88.128.0 – 101.88.255.255 101.89.0.0 – 101.89.129.0	101.88.128.0	101.88.255.255	101.88.128.1 – 101.88.255.254

For 200.20.20.189/23:

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

SO₂

Subnet Mask	Network Range	Network IP	Broadcast IP	Valid Host IP Range
255.255.254.0	200.20.0.0 – 200.20.1.255	200.20.20.0	200.20.21.255	200.20.20.1 – 200.20.21.254
	200.20.2.0 – 200.20.3.255			
	200.20.4.0 – 200.20.5.255			
	200.20.6.0 – 200.20.7.255			
	200.20.8.0 – 200.20.9.255			
	200.20.10.0 – 200.20.11.255			
	200.20.12.0 – 200.20.13.255			
	200.20.14.0 – 200.20.15.255			
	200.20.16.0 – 200.20.17.255			
	200.20.18.0 – 200.20.19.255			
	200.20.20.0 – 200.20.21.255			
	200.20.22.0 – 200.20.23.255			

Practice Question

🔗 **Problem1:** Briefly describe **CSMA**. What should a station do if the channel is busy? Explain the **different approaches of CSMA** and state which method is **more resistant to collisions**.

➤ **Solutions:**

⇒ CSMA is a multiple access protocol where each station **listens to the channel before transmitting**.

The idea: Before sending, each station **listens (senses)** the channel to check if it's free. This helps reduce collisions compared to ALOHA.

➤ **What should a station do if the channel is busy?**

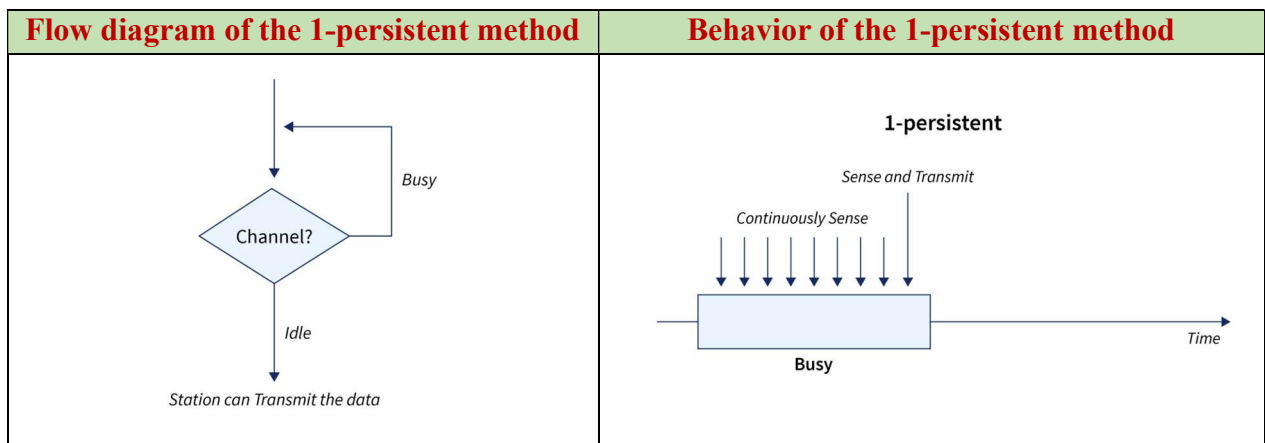
⇒ If the channel is busy, the station must **wait** and use a specific method (persistent or non-persistent) to decide when to try again.

➤ **Different Approaches of CSMA:**

1. 1-Persistent CSMA
2. Non-Persistent CSMA
3. P-Persistent CSMA

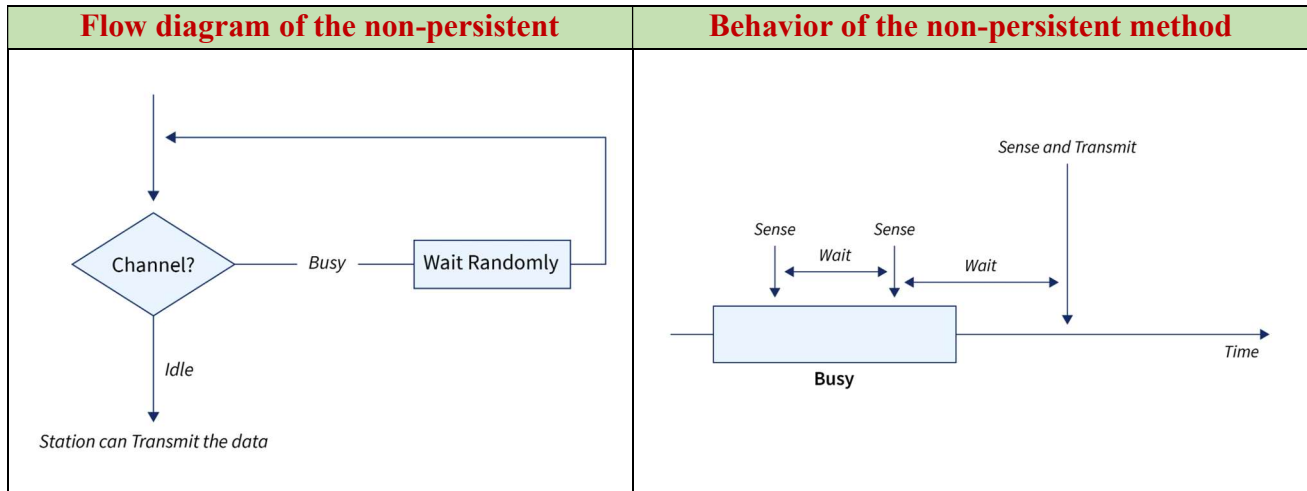
1-Persistent CSMA

- ⇒ Sense the channel **continuously**
- ⇒ if channel is **idle**, send immediately (probability = 1).
- ⇒ If channel is **busy**, keep sensing until it's free.
- ⇒ **Problem:** High chance of collision since many station may send instantly.



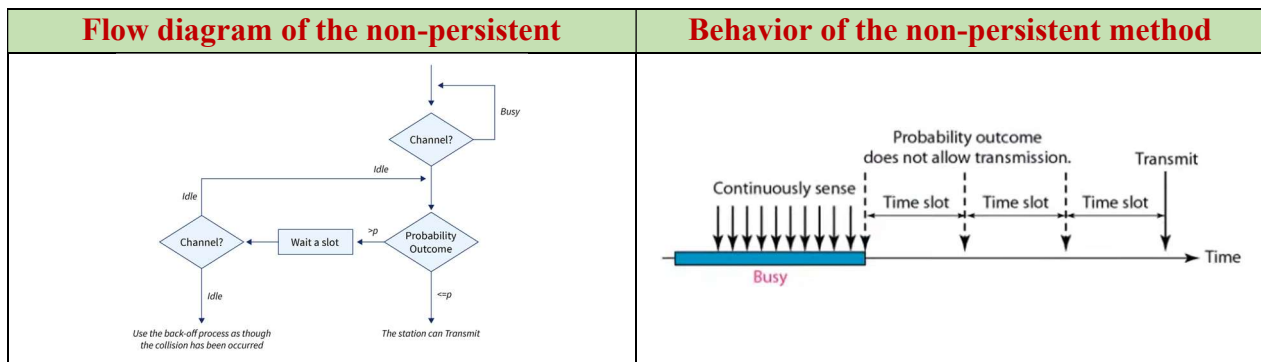
Non-Persistent CSMA

- ⇒ Sense the channel, if channel is **busy**, wait a **random time** before checking again.
- ⇒ Advantage: Fewer collisions.
- ⇒ **Disadvantage**: Lower efficiency (channel may stay idle unnecessarily).



P-Persistent CSMA

- ⇒ Used in **slotted channels**.
- ⇒ If channel is **idle**:
 - Send with probability **p**.
 - Wait for next slot with probability **1 - p**.
- ⇒ Combines the benefits of the above two methods.



➤ Which method is most resistant to collisions?

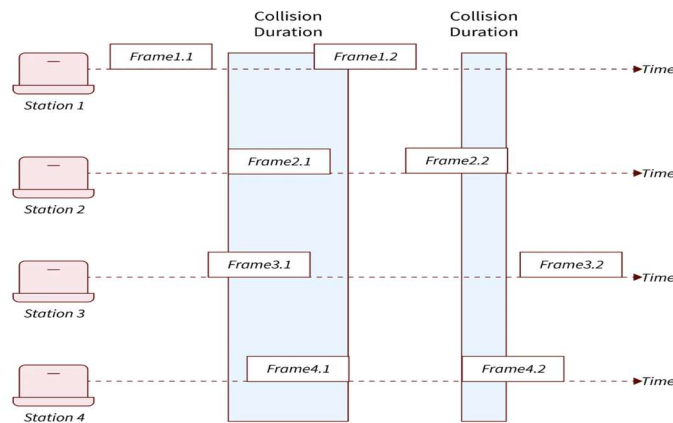
- ⇒ **Non-Persistent CSMA**: Because stations **wait a random amount of time** when the channel is busy, reducing the chance that multiple stations resend at the same moment.

✧ **Problem2:** Explain the **Pure ALOHA** architecture with a diagram. Identify the **vulnerable time** in Pure ALOHA.

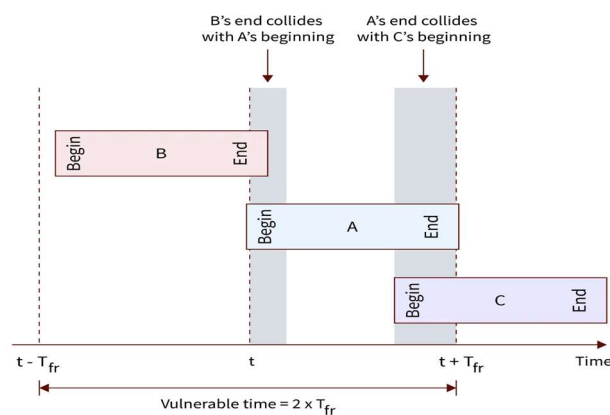
➤ **Solution:**

- ⇒ Each station sends data **whenever it wants**.
- ⇒ If two stations send data at the same time → **collision** happens.
- ⇒ Sender waits for **acknowledgment (ACK)** from the receiver.
- ⇒ If no ACK is received (because of collision), it waits for a **random backoff time** before resending.
- ⇒ After several failed attempts (Kmax), it stops trying for a while.
- ⇒ **Efficiency:** Low (about 18%).

➤ **Collisions in Shared Channel:**



➤ **Vulnerable Time for Pure Aloha:**



$$\text{Vulnerable time} = 2 \times T_{fr}$$

✂ **Problem3:** Suppose there are six users A, B, C, D, E and F. They want to send bit sequence +1, 0, +1, +1, -1 & -1, respectively. Generate the code for each user and find the transmit sequence. Suppose that a user G wants to decode the information sent from the user B, perform the necessary steps and determine B's transmitted symbol.

Is given,

6 User = A, B, C, D, E, F.

We need Code = $c^1, c^2, c^3, c^4, c_5, c_6$. [Since the number of codes is 6, use W_8]

data bits = $d_1 = +1, d_2 = 0, d_3 = +1, d_4 = +1, d_5 = -1, d_6 = -1$

So, the data on the channel will be:

$$data = (d_1c_1 + d_2c_2 + d_3c_3 + d_4c_4 + d_5c_5 + d_6c_6) \dots \dots \dots (i)$$

So, generate the code,

$$W_1 = [+1]$$

$$W_2 = \begin{bmatrix} W_1 & W_1 \\ W_1 & \bar{W}_1 \end{bmatrix} = \begin{bmatrix} +1 & +1 \\ +1 & +1 \end{bmatrix} = \begin{bmatrix} +1 & +1 \\ +1 & -1 \end{bmatrix}$$

$$W_4 = \begin{bmatrix} W_2 & W_2 \\ W_2 & \bar{W}_2 \end{bmatrix} = \begin{bmatrix} +1 & +1 & +1 & +1 \\ +1 & -1 & +1 & -1 \\ +1 & +1 & +1 & +1 \\ +1 & -1 & +1 & -1 \end{bmatrix} = \begin{bmatrix} +1 & +1 & +1 & +1 \\ +1 & -1 & +1 & -1 \\ +1 & +1 & -1 & -1 \\ +1 & -1 & -1 & +1 \end{bmatrix} = \begin{matrix} c_1 \\ c_2 \\ c_3 \\ c_4 \end{matrix}$$

$$W_8 = \begin{bmatrix} W_4 & W_4 \\ W_4 & \bar{W}_4 \end{bmatrix} = \begin{bmatrix} +1 & +1 & +1 & +1 & +1 & +1 & +1 & +1 \\ +1 & -1 & +1 & -1 & +1 & -1 & +1 & -1 \\ +1 & +1 & -1 & -1 & +1 & +1 & -1 & -1 \\ +1 & -1 & -1 & +1 & +1 & -1 & -1 & +1 \\ +1 & +1 & +1 & +1 & -1 & -1 & -1 & -1 \\ +1 & -1 & +1 & -1 & -1 & +1 & -1 & +1 \\ +1 & +1 & -1 & -1 & -1 & -1 & +1 & +1 \\ +1 & -1 & -1 & +1 & -1 & +1 & -1 & -1 \end{bmatrix} = \begin{matrix} c_1 \\ c_2 \\ c_3 \\ c_4 \\ c_5 \\ c_6 \\ c_7 \\ c_8 \end{matrix}$$

Now put the values in equation (i),

$$data = (d_1c_1 + d_2c_2 + d_3c_3 + d_4c_4 + d_5c_5 + d_6c_6)$$

$$data = \begin{matrix} +1 \times \\ +0 \times \\ +1 \times \\ +1 \times \\ -1 \times \\ -1 \times \end{matrix} \begin{bmatrix} +1 & +1 & +1 & +1 & +1 & +1 & +1 & +1 \\ +1 & -1 & +1 & -1 & +1 & -1 & +1 & -1 \\ +1 & +1 & -1 & -1 & +1 & +1 & -1 & -1 \\ +1 & -1 & -1 & +1 & +1 & -1 & -1 & +1 \\ +1 & +1 & +1 & +1 & -1 & -1 & -1 & -1 \\ +1 & -1 & +1 & -1 & -1 & +1 & -1 & +1 \end{bmatrix}$$

$$data = \begin{bmatrix} +1 & +1 & +1 & +1 & +1 & +1 & +1 & +1 \\ +0 & -0 & +0 & -0 & +0 & -0 & +0 & -0 \\ +1 & +1 & -1 & -1 & +1 & +1 & -1 & -1 \\ +1 & -1 & -1 & +1 & +1 & -1 & -1 & +1 \\ -1 & -1 & -1 & -1 & +1 & +1 & +1 & +1 \\ -1 & +1 & -1 & +1 & +1 & -1 & +1 & -1 \end{bmatrix}$$

$$data = [+1 \quad +1 \quad -3 \quad +1 \quad +5 \quad +1 \quad +1 \quad +1]$$

$$\therefore \text{Transmit sequence data} = [+1 \quad +1 \quad -3 \quad +1 \quad +5 \quad +1 \quad +1 \quad +1]$$

Now, since User G wants to receive data from user B:

⇒ We must multiply the received data on the channel by c_2 (code of User 2):

$$\text{Transmit data} \times c_2 = [+1 \quad +1 \quad -3 \quad +1 \quad +5 \quad +1 \quad +1 \quad +1] \times$$

$$[+1 \quad -1 \quad +1 \quad -1 \quad +1 \quad -1 \quad +1 \quad -1]$$

$$data' = 1 - 1 - 3 - 1 + 5 - 1 + 1 - 1 = 0$$

Now divide by 8:

$$data' = \frac{0}{8} = 0$$

Therefore, User G gets the data of User B correctly

🔗 **Problem4:** Explain the **FDMA & TDMA** architecture with a diagram & Example.

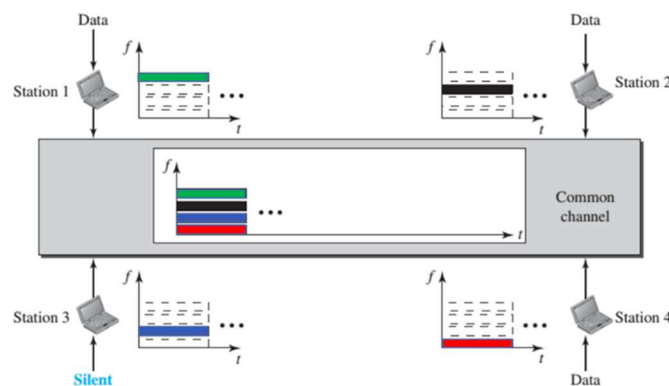
➤ **Solution:**

❖ **FDMA (Frequency Division Multiple Access):**

- ⇒ The total bandwidth is divided into **different frequency bands**.
- ⇒ Each station gets its **own frequency band** all the time.
- ⇒ **Guard bands** (small empty spaces) are added to prevent overlap and interference.

➤ **Example:** If there are 3 users, each gets a separate frequency to talk at the same time.

➤ **Used in:** Radio, Satellite, and Cellular systems.

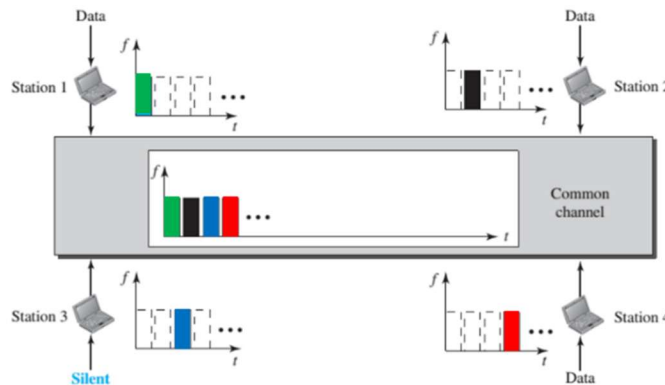


❖ **TDMA (Time Division Multiple Access)**

- ⇒ The whole channel is used by **each station one at a time**.
- ⇒ Each station gets a **time slot** to send data.
- ⇒ **Synchronization** is needed so each station knows when its time slot starts.

➤ **Example:** Like people taking turns to talk — one after another.

➤ **Used in:** GSM mobile systems.



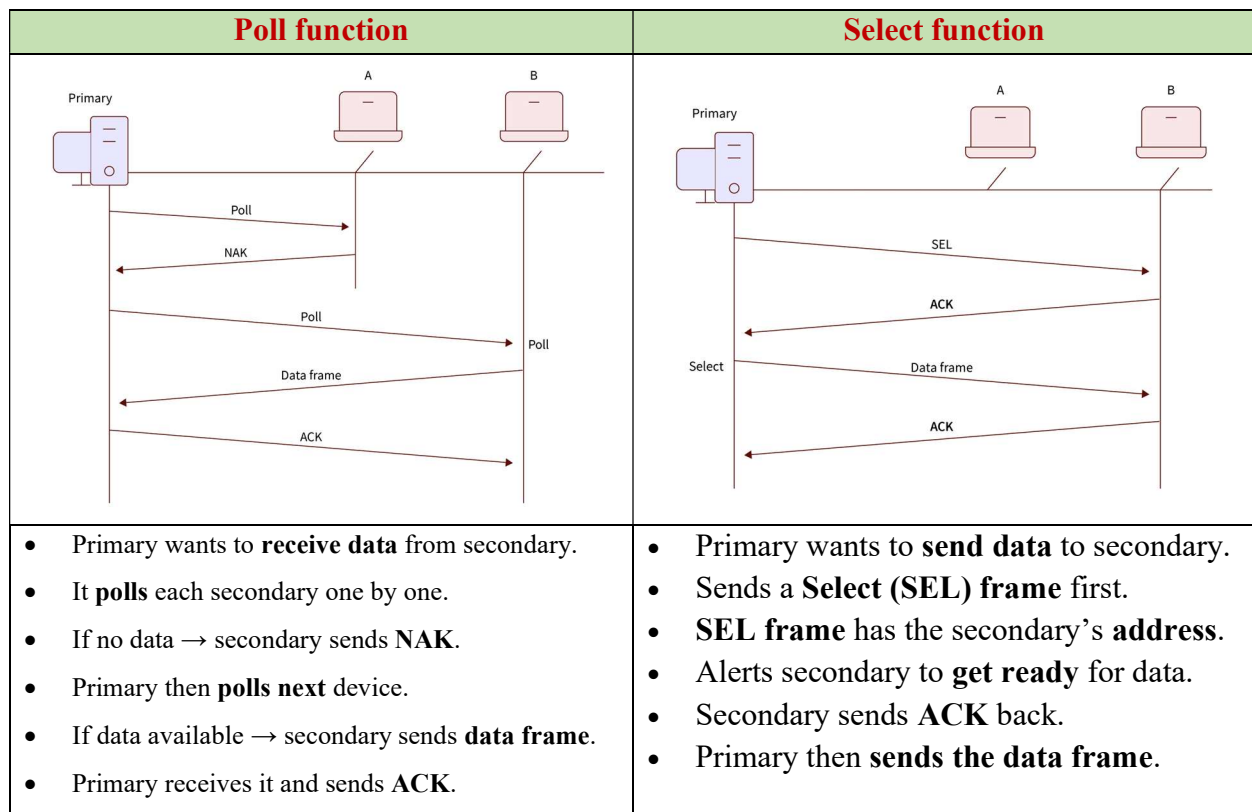
✂ **Problem5:** Explain the **Polling architecture** with the help of a **diagram**. Clearly describe how the **Poll function** and the **Select function** work in this architecture.

➤ **Solution:**

- ⇒ A **primary station (controller)** controls communication.
- ⇒ **Secondary stations** can only send/receive when allowed by **primary station**.
- ⇒ Two operations:
 - **Poll function:** Primary asks if secondary has data to send.
 - **Select function:** Primary tells secondary to receive data.

➤ **Example:** Like a teacher asking each student one by one to speak.

➤ **Used in:** master–slave or client–server setups.



✂ **Problem6:** What are the **different approaches of Controlled Access Protocols**? Explain how the **Reservation architecture** works with the help of a **diagram**.

➤ **Solution:**

➤ **Different Approaches of Controlled Access Protocols:**

1. **Reservation**
2. **Polling**
3. **Token Passing**

❖ Reservation:

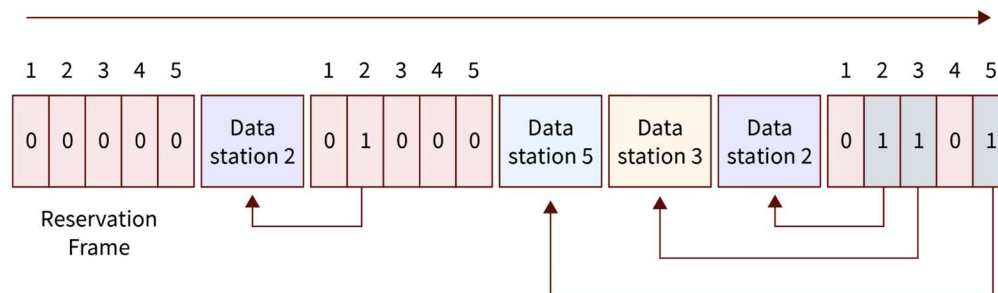
- ⇒ Each station **reserves** the channel before data transmission.
- ⇒ The time is divided into **slots**; a station sends a **reservation message** to claim a slot.
- ⇒ Common in systems where predictable access is needed (e.g., satellite networks).

How it works:

- If there are **N stations** in the system, there are exactly **N reservation minislots** in the reservation frame.
- Each minislot belongs to a station.
- When a station needs to send a data frame, it makes a reservation in its own minislot.
- The stations that have made reservations can send their data frame after the reservation frame.

➤ **Example:** Like booking a seat before watching a movie — once reserved, you get your turn.

Reservation access method:

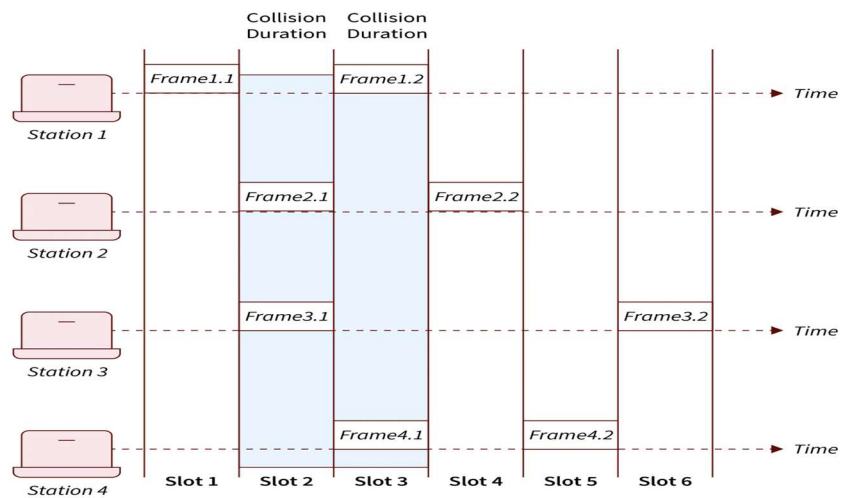


✈ **Problem7:** Explain the **Slotted ALOHA** architecture with a diagram. Identify the **vulnerable time** in Slotted ALOHA.

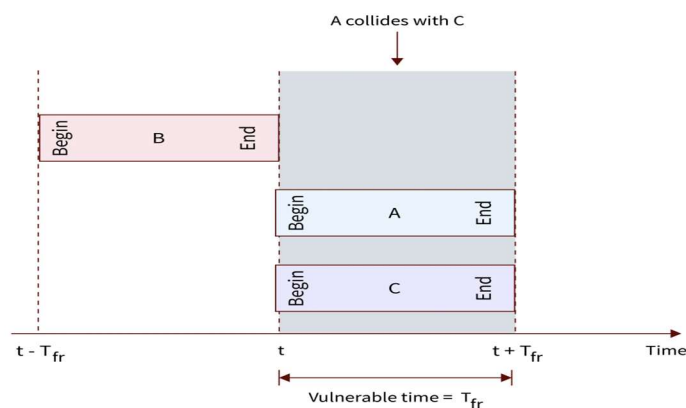
➤ **Solution:**

- ⇒ Time is divided into **slots** (each slot = frame transmission time).
- ⇒ Stations can only send **at the start** of a slot.
- ⇒ Reduces collision chances.
- ⇒ **Vulnerable time** = T_{fr} .
- ⇒ **Efficiency:** Higher than Pure ALOHA (about 36%).

➤ **Collisions in Shared Channel:**



➤ **Vulnerable Time for Pure Aloha:**



Vulnerable time = T_{fr}